

Republic of Tunisia
Ministry of Higher Education and Scientific
Research

General Directorate
of Technological Studies

Higher Institute
of Technological Studies of Gafsa



الجمهورية التونسية
وزارة التعليم العالي والبحث العلمي
الإدارة العامة للدراسات التكنولوجية
المعهد العالي للدراسات التكنولوجية
بقفصة

Department of Information Technology

TI /J26

قسم تكنولوجيا
الإعلامية

Final Year Project

In order to obtain the

Bachelor's Degree in Information Technology.

Option: Computer System Development

Title

Design and Development of the Predictra Threat Map

**Real-Time Cyber Threat Intelligence Aggregation and
Spatial Analytics Platform**

Carried out by

Brahim JABALLI

Chiheb AMRI

Presented on 18/06/2026 before the jury composed of:

Ms. Hadhami ISSAOUI

President (ISET Gafsa)

Mr. Walid Hammemi

Reporter (ISET Gafsa)

Mr. Anis DHAHRI

Academic Supervisor (ISET Gafsa)

Academic Year: 2025/2026

DEDICATION

To the people who have supported me throughout my academic journey,
And contributed, directly or indirectly, to the completion of this project.

To the memory of my late father,
Whose love and guidance have profoundly shaped who I am today.

To my mother,
For her unconditional love, patience, and continuous support, which have
been a constant source of motivation throughout my studies.

To my elder sister and my younger brother,
For their encouragement and support during this journey.

To my best friend,
For his valuable support, encouragement, and presence throughout both the
difficult and successful moments of this experience.

To all those who have contributed,
Directly or indirectly, to my success and to the completion of this work.

Brahim Jaballi

DEDICATION

I dedicate this work with profound love and gratitude to:

To the memory of my beloved Father,

Who is no longer with us, but whose spirit, wisdom, and dreams for me continue to light my path. I hope this achievement makes you proud in heaven. You will forever remain my greatest source of inspiration and strength.

To my precious Mother,

My pillar of support, hope, and tenderness. Thank you for your endless sacrifices, your constant prayers, and your unconditional love that guided me through every step of this journey. This success is yours.

To my brothers, sisters, and the cherished souls who complete our family,

For your warmth, continuous encouragement, and for always being my anchor. Your belief in me has been my greatest motivation.

To my dear Friends,

Who stood by me, shared the late-night struggles, the laughter, and the memories, making this path all the more meaningful.

Chiheb Amri

Acknowledgments

We would like to express our deepest gratitude and sincere appreciation to all those who contributed to the successful completion of this graduation project (Projet de Fin d'Études).

First and foremost, we wish to extend our profound gratitude to our academic supervisor, **Mr. Anis Dhahri**, as well as the faculty members of the Department of Information Technology at **ISET Gafsa**. Their constructive feedback, academic guidance, and rigorous standards have significantly shaped our engineering mindset and technical methodology.

We are immensely grateful to our host company, **Predictra**, and its visionary engineering team for welcoming us into their innovative workspace. Our sincere thanks go to our professional mentors at Predictra for providing us with the resources, bleeding-edge datasets, and operational insights needed to design and implement the central Threat Map platform. Their constant feedback was invaluable in aligning the platform with the demands of real-world security operations centers.

We also thank the distinguished members of the jury who have agreed to review and evaluate this graduation thesis. We are honored by their time, critique, and assessment.

Finally, we owe our deepest gratitude to our families, parents, and friends. Their unwavering love, patience, moral support, and endless sacrifices have been our absolute foundation of strength throughout our student years and during the execution of this project.

Brahim Jaballi

Chiheb Amri

Contents

General Introduction	1
1 General Project Framework and Methodology	4
1.1 Introduction	4
1.2 Host Organization: Predictra	4
1.2.1 Company Profile and Mission	4
1.2.2 Organizational Structure	5
1.2.3 Inter-Departmental Workflows	5
1.3 Project Context and Problem Definition	6
1.4 State of the Art: Comparative Analysis of CTI Solutions	6
1.4.1 Open-Source Solutions	6
1.4.2 Commercial Solutions	7
1.4.3 Comparative Summary Table	7
1.5 Critique of Existing Systems and Proposed Solution	8
1.6 Work Methodology and Modeling Standards	8
1.6.1 Agile Scrum Framework	9
1.6.2 Twice-Weekly Scrum Coordination and Technical Review Rituals	11
1.6.3 UML Modeling Standards	13
1.7 Conclusion	13
2 Project Specification, Planning, and Architecture	14
2.1 Introduction	14
2.2 Requirements Specification	14
2.2.1 Functional Requirements (RF)	14
2.2.2 Non-Functional Requirements (RNF)	19
2.3 System Needs Modeling	20
2.3.1 Identification of Actors	20
2.3.2 Global Use Case Diagram	20
2.4 Project Management with Scrum	23
2.4.1 Product Backlog	23
2.4.2 Previsional Gantt Chart	23
2.5 Work Environment	23
2.5.1 Hardware Environment	24
2.5.2 Software Environment and Development Tools	24
2.5.3 Stack and Frameworks Comparative Analysis	24
2.6 Architectural Design	26
2.6.1 Deployment Architecture	26
2.6.2 System Package Component Architecture	27

2.6.3	Application Logical Architecture	27
2.6.4	Global Class Diagram	28
2.7	Conclusion	29
3	First Release - Core Ingestion and 3D Visualization	30
3.1	Introduction	30
3.2	Sprint 1: Data Ingestion and Enrichment Pipeline	30
3.2.1	Sprint Backlog and Objectives	30
3.2.2	Sprint Analysis: Use Case Modeling	30
3.2.3	Sprint Conception: Classes, Sequences, and Schemas	31
3.2.4	Sprint Realization: Detailed Scraper Implementations	34
3.2.5	Sprint Realization: Ingestion & Enrichment Logic	41
3.3	Sprint 2: Real-Time SSE Stream and 3D Globe Visualization	42
3.3.1	Sprint Backlog and Objectives	42
3.3.2	Sprint Analysis: Use Case Modeling	42
3.3.3	Sprint Conception: Trigonometric Geocoding and Arc Interpolation	43
3.3.4	Sprint Realization: Shaders, Shards, and WebGL Globe	44
3.4	Conclusion	46
4	Second Release - STIX Analytics and Performance Guardrails	47
4.1	Introduction	47
4.2	Sprint 3: STIX 2.1 Threat Analysis Dashboard	47
4.2.1	Sprint Backlog and Objectives	47
4.2.2	Sprint Analysis: Use Case Modeling	48
4.2.3	Sprint Conception: Parser Interfaces and Graph Sequences	48
4.2.4	Sprint Realization: STIX Ingestion and Mapping	51
4.3	Sprint 4: Analytics, History, and Performance Polishing	52
4.3.1	Sprint Backlog and Objectives	52
4.3.2	Sprint Analysis: Use Case Modeling	53
4.3.3	Sprint Conception: Search and Performance Buffers	53
4.3.4	Performance Stabilization Mathematics	54
4.3.5	Sprint Realization: Performance Guardrails and Buffers	55
4.4	System Project Retrospective	56
4.4.1	Scrum Development Analytics	56
4.4.2	Overcoming Critical Technical Challenges	57
4.4.3	Architectural Scalability and Future Improvements	57
4.5	Conclusion	58
5	Application Interfaces	59
5.1	Live Threat Map	59
5.2	System Dashboard	59
5.3	STIX Intelligence Workspace	60
5.4	Historical Threat Browser	61
5.5	Global Analytics	61
	Appendices	63
	General Conclusion	87

List of Figures

1.1	Organizational structure of Predictra.	5
1.2	Agile Scrum development lifecycle flowchart.	10
2.1	UML Global Use Case diagram for the Predictra CTI platform.	21
2.2	Previsional Gantt Chart of development sprints.	23
2.3	UML Component diagram of the Predictra CTI system.	27
2.4	Predictra platform multi-tier layered application architecture.	28
2.5	UML Global Class diagram of the Predictra CTI system.	29
3.1	Sprint 1: Use Case diagram for threat data harvesting.	31
3.2	Sprint 1: Conception Class diagram.	32
3.3	Sprint 1: Sequence diagram of the harvesting and enrichment loop.	34
3.4	Sprint 2: Use Case diagram for the 3D map visualizer.	43
3.5	Sprint 2: Sequence diagram of real-time streaming and WebGL rendering.	44
4.1	Sprint 3: Use Case diagram for the STIX 2.1 dashboard.	48
4.2	Sprint 3: Conception Class diagram.	51
4.3	Sprint 3: Sequence diagram of STIX bundle parsing and graph rendering.	51
4.4	Sprint 4: Use Case diagram for database browsing and performance monitoring.	53
4.5	Sprint 4: Conception Class diagram.	54
4.6	Sprint 4: Sequence diagram of paginated historical threat search.	54
5.1	The 3D Live Threat Map.	59
5.2	The System Dashboard.	60
5.3	The STIX 2.1 Workspace.	60
5.4	The Historical Threat Browser.	61
5.5	The Analytics Dashboard.	62

List of Tables

1.1	Academic comparative analysis of global CTI solutions.	8
1.2	Operational breakdown of bi-weekly coordination meetings held over 6 months.	13
2.1	Detailed descriptions of the global primary use cases and their relationships.	22
2.2	Comprehensive architectural comparison of selected vs alternative technologies.	26
2.3	Frontend Reverse-Proxy Routing Configuration.	26
3.1	Sprint 1: Use Case descriptions.	31
3.2	MongoDB BSON Schema Dictionary ('models/ThreatEvent.js').	33
3.3	Checkpoint Scraper Endpoint Specifications.	34
3.4	Checkpoint Raw Threat Payload Schema.	35
3.5	Bitdefender Scraper Endpoint Specifications.	35
3.6	Bitdefender Raw Threat Payload Schema.	36
3.7	Fortinet Scraper Endpoint Specifications.	36
3.8	Fortinet Raw Threat Payload Schema.	36
3.9	URLhaus Scraper Endpoint Specifications.	37
3.10	URLhaus Raw Threat Payload Schema.	37
3.11	Kaspersky Feodo Scraper Endpoint Specifications.	38
3.12	Kaspersky Feodo Raw Threat Payload Schema.	38
3.13	AlienVault Scraper Endpoint Specifications.	39
3.14	AlienVault Raw Threat Payload Schema.	39
3.15	Ransomwatch Scraper Endpoint Specifications.	39
3.16	Ransomwatch Raw Threat Payload Schema.	40
3.17	MontySecurity C2 Scraper Endpoint Specifications.	40
3.18	MontySecurity C2 Raw Threat Payload Schema.	40
3.19	MISP Galaxy Scraper Endpoint Specifications.	41
3.20	MISP Galaxy Raw Threat Payload Schema.	41
3.21	Sprint 2: Use Case descriptions.	43
3.22	Atmospheric Glow Shader Parameters and Interpolated Variables.	45
4.1	Sprint 3: Detailed Use Case descriptions.	48
4.2	STIX 2.1 Sample Intelligence Bundle Components.	50
4.3	STIX 2.1 Parser Object Classification Rules.	52
4.4	Sprint 4: Use Case descriptions.	53
4.5	Adaptive Event Sampling Engine control parameters.	55
4.6	Circular RingBuffer API Specifications.	55
4.7	FPS Telemetry Collector State Variables.	56

List of Abbreviations

API	Application Programming Interface
APT	Advanced Persistent Threat
ASN	Autonomous System Number
C2	Command and Control
CDN	Content Delivery Network
CISA	Cybersecurity and Infrastructure Security Agency
CORS	Cross-Origin Resource Sharing
CS	Cybersecurity
CTI	Cyber Threat Intelligence
DOM	Document Object Model
DVR	Digital Video Recorder
EDR	Endpoint Detection and Response
FPS	Frames Per Second
GLSL	OpenGL Shading Language
HUD	Heads-Up Display
IOC	Indicator of Compromise
IP	Internet Protocol
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
MISP	Malware Information Sharing Platform
OS	Operating System
PFE	Projet de Fin d'Études (Graduation Project)
R3F	React Three Fiber
RaaS	Ransomware-as-a-Service
RDAP	Registration Data Access Protocol
REST	Representational State Transfer
SIEM	Security Information and Event Management
SOC	Security Operations Center
SPA	Single Page Application

SSE	Server-Sent Events
STIX	Structured Threat Information Expression
TTP	Tactics, Techniques, and Procedures
TTL	Time-To-Live
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
VM	Virtual Machine
WebGL	Web Graphics Library
WHOIS	Protocol for querying databases that store registered users
WS	WebSocket
XDR	Extended Detection and Response

General Introduction

Context of the Global Cybersecurity Landscape

In the modern digital era, the hyper-connectivity of corporate networks, cloud platforms, and critical infrastructures has created an unprecedented and highly volatile attack surface. Modern organizations rely entirely on software and network availability to operate, which has made them highly attractive targets for malicious actors. The threat landscape has evolved from simple, opportunistic scripts and virus files into highly coordinated, structured, and persistent adversary campaigns [1].

These campaigns are led by a diverse array of threat actors, including sophisticated groups conducting digital espionage, ransomware cartels seeking financial extortion, and hacktivists launching disruptive operations. Adversaries routinely exploit security flaws, set up command servers to control attacks, and leak stolen corporate documents. In this hostile landscape, traditional defenses (such as firewalls and antivirus software) are no longer enough to protect sensitive assets.

The Critical Problematic: Blindness to the Adversary

A foundational axiom of strategic warfare, articulated by the legendary Chinese military strategist Sun Tzu in *The Art of War* [3], states:

"If you know the enemy and know yourself, you need not fear the result of a hundred battles. If you know yourself but not the enemy, for every victory gained you will also suffer a defeat. If you know neither the enemy nor yourself, you will succumb in every battle."

When applied to modern cybersecurity, the vast majority of security programs focus exclusively on the second clause of Sun Tzu's axiom: **"knowing themselves"**. Organizations allocate significant resources to internal defense mechanisms. They install detection agents on laptops, aggregate internal logs in monitoring systems, and configure access controls.

While these tools are essential for monitoring internal systems, they leave the organization blind to **"the enemy"**. Defenders have no external visibility. They do not know who is targeting their specific sector, which malicious servers are active, what scans are targeting their networks, or if their data is being traded online. Operating in this state leaves security teams entirely reactive, forced to wait until a breach occurs before responding.

The Solution: Proactive Threat Intelligence (CTI)

To resolve this imbalance, the field of **Cyber Threat Intelligence (CTI)** has emerged. CTI is the collection and analysis of data regarding cyber threats. It bridges the visibility gap by providing defenders with **external awareness**, answering critical questions:

- **Who is the threat?** (Identifying actor profiles and historical campaign records).
- **What are their tactics and methods?** (Understanding adversary techniques using frameworks like the MITRE ATT&CK matrix, which outlines the phases of an attack).
- **What infrastructure are they using?** (Tracking active malicious servers, phishing domains, and malware download URLs).

By integrating threat intelligence, defenders transition from a reactive posture to a proactive defense, identifying and blocking incoming threats before they execute.

The Metric of Failure: Compromise Dwell Time

The severe operational cost of defending a network without external threat visibility is illustrated by global security metrics. According to industry reports, the global average **dwell time** – the duration between an initial compromise and its detection – consistently exceeds **200 days** (approximately 6 to 7 months).

During this long window, attackers move through internal networks, gain access to administrative controls, steal intellectual property, and deploy ransomware. The main cause of this long dwell time is that defenders only detect compromises after an internal alert is triggered. By aggregating and streaming external threat intelligence in real-time, organizations can identify attacks immediately, reducing detection time from months to seconds.

Objective of the Graduation Project

The primary objective of this graduation project (Projet de Fin d'Études) is to design and implement the **Predictra Threat Map**, a threat intelligence platform. The system aggregates live threat indicators from 9+ external feeds (covering malware sites, malicious servers, and ransomware leaks), finds their geographical locations, classifies target business sectors, and streams these threats in real-time.

On the frontend, these threat flows are displayed on an interactive 3D virtual Earth, allowing teams to instantly see attack vectors. Furthermore, the platform integrates a STIX 2.1 [2] file parser and a MITRE ATT&CK mapping grid to support threat analysis.

Structure of the Graduation Report

To detail the research, modeling, and realization of the Predictra Threat Map, this report is structured into four distinct chapters:

- **Chapter 1: General Project Framework and Methodology** introduces our host company Predictra, reviews existing threat intelligence platforms, and details our Scrum and UML development methodologies.
- **Chapter 2: Project Specification, Planning, and Architecture** documents functional and non-functional requirements, defines system actors, breaks down the backlog, and maps out the system architecture.
- **Chapter 3: First Release - Core Ingestion and 3D Visualization** details the design and implementation of Sprint 1 (our backend scrapers and target classification engine) and Sprint 2 (our real-time stream and 3D globe visualizer).
- **Chapter 4: Second Release - STIX Analytics and Performance Guardrails** presents Sprint 3 (our STIX 2.1 parser and MITRE matrix) and Sprint 4 (our searchable logs browser, "Target My IP" locator, circular memory buffers, and frame rate optimization engine).

Chapter 1

General Project Framework and Methodology

1.1 Introduction

To establish a clear, academically rigorous roadmap for the design, development, and realization of the **Predictra Threat Map**, it is essential to first understand the organizational environment, project context, and state of the art in the threat intelligence domain. This chapter introduces our host company, details the commercial and open-source landscape of Cyber Threat Intelligence (CTI) platforms, critiques existing market solutions, and defines the Agile Scrum framework and UML standards adopted as our development and modeling methodologies.

1.2 Host Organization: Predictra

1.2.1 Company Profile and Mission

This graduation project was hosted by **Predictra**, a next-generation cybersecurity technology firm specializing in AI-driven Cyber Threat Intelligence (CTI). Predictra's core vision is to democratize threat intelligence, converting millions of raw, distributed cybersecurity data points into highly structured, real-time, and visually actionable insights.

Predictra provides enterprise security teams, Security Operations Centers (SOCs), and threat researchers with advanced reconnaissance capabilities, brand monitoring, ransomware leak tracking, and infrastructure threat analysis. The host company represents a modern threat-focused corporate environment, providing access to massive telemetry datasets and industrial expertise in cyber adversary tracking.

Predictra's platforms leverage advanced machine learning models to ingest unstructured data from public channels, dark web marketplaces, chat platforms, and open-source repositories. The goal of the organization is to equip modern security departments with proactive defenses, shifting the security paradigm from reactive incident response to predictive threat preemption. By offering real-time visibility into adversary activities, Predictra empowers companies to actively block threats before they breach internal network perimeters.

1.2.2 Organizational Structure

To support its mission of providing real-time global intelligence, Predictra is structured into three highly specialized departments, as modeled in Figure 1.1.



Figure 1.1: Organizational structure of Predictra.

The specific responsibilities of these departments are detailed below:

- **Threat Research Department:** Composed of security analysts and researchers. This department focuses on analyzing cyber campaigns, monitoring extortion websites, and maintaining threat databases. They validate threat indicators and create signatures. Their operations involve examining malware samples, tracking attacker networks, and cataloging actor tactics. Their output is threat intelligence used to feed the ingestion platforms.
- **Security Engineering Department:** Composed of software and database developers. This department is responsible for building ingestion pipelines, data streaming, and the user interface. They focus on database performance, user experience, and cloud deployments. They coordinate with researchers to support new threat indicators.
- **Sales and Operations Department:** Manages client relations, enterprise deployments, marketing campaigns, and commercial threat briefing dissemination, ensuring the platform's features align with commercial market requirements. They gather user feedback from enterprise customers, identify market gaps, and coordinate training and technical support services.

1.2.3 Inter-Departmental Workflows

To maintain a high velocity of threat detection and update dissemination, these three departments coordinate via highly structured workflows. When the Threat Research team identifies a new ransomware leak site or an active botnet command-and-control server cluster, they compile the technical signatures and pass them to the Security Engineering team.

The engineers design new scrapers or update the existing classification databases to support these indicators. Once validated, the engineering team deploys the updates to

production environments. Finally, the Sales and Operations team briefs enterprise clients on the newly integrated monitoring capabilities, gathering immediate operational feedback to guide future development iterations.

1.3 Project Context and Problem Definition

Historically, analyzing external threat feeds was a slow, manual process. Threat intelligence teams had to poll separate tabular feeds, manually normalize indicators, and compile static briefings. This tabular approach creates a high **compromise dwell time** – often exceeding 200 days – during which adversaries move laterally undetected. Furthermore, Security Operations Center (SOC) analysts suffer from visual fatigue when looking at thousands of static text logs in spreadsheets or simple dashboards. They lack an intuitive, real-time spatial visual interface that provides immediate, global threat situational awareness at a glance.

The **Predictra Threat Map** was conceived to bridge this critical operational gap, transforming high-velocity, multi-source external cyber threat data into a live, interactive, 3D visual intelligence map and structured analysis dashboard.

1.4 State of the Art: Comparative Analysis of CTI Solutions

To design a robust CTI platform, we conducted a rigorous comparative analysis of existing open-source and commercial threat intelligence solutions, evaluating their data models, stream latencies, spatial visualization support, and hardware footprints.

1.4.1 Open-Source Solutions

MISP (Malware Information Sharing Platform)

MISP is a widely used open-source platform for sharing threat information. It supports storing and tagging IP addresses, file hashes, and domains.

From a technical perspective, MISP uses standard database structures. While it is highly capable at storing large lists of historical threats, it is not optimized for real-time streaming. Analysts must request updates periodically, which introduces delays. Furthermore, the user interface consists primarily of text-based tables and lacks real-time 3D visual maps.

Maintaining a MISP instance requires regular management. Security teams must allocate engineering resources to monitor the system and refine database rules. Additionally, because the interface is complex, security teams must spend time training analysts before using it in security operations.

OpenCTI (Open Cyber Threat Intelligence)

OpenCTI is a modular platform designed to represent threat intelligence as a graph, linking actors, campaigns, malware, and targeted sectors. It uses specialized databases for indexing and searching, and features connectors to import public and private threat feeds.

However, OpenCTI's architecture is heavy. Running the platform requires managing multiple service layers, which demands high-specification servers and increases operational costs. Furthermore, the interface is optimized for long-term analytical research rather than real-time visual tracking.

From a hosting perspective, the server requirements mean that organizations must allocate significant hardware resources. For smaller teams, this makes the hosting costs high, reducing the financial benefits of using open-source software.

1.4.2 Commercial Solutions

Recorded Future

Recorded Future is a major commercial platform that gathers threat indicators from web, forum, and social media channels. It translates these pages into standardized profiles, offering threat scoring and technical indicators, with integrations for major security tools.

Despite its capabilities, Recorded Future is highly expensive, making it inaccessible to smaller organizations. Furthermore, the dashboard is focused on text reports and simple static 2D maps, lacking real-time animations for display screens.

Because it is a closed system, organizations must rely entirely on the vendor's algorithms. Integrating custom data sources not supported by default can be difficult and costly.

Cyble

Cyble is a commercial provider focusing on brand protection and monitoring online forums and chat channels for leaked credentials and assets. It provides automated alerts and risk indicators.

Like other commercial options, Cyble requires subscription fees and does not allow custom developer integrations. Its interfaces are designed as static reports, lacking real-time animations, which limits its usefulness for active display environments.

1.4.3 Comparative Summary Table

To clarify the distinctions, a comparative matrix of key platform parameters is detailed in Table 1.1.

Feature / Metric	MISP		OpenCTI		Recorded Future	Predictra Threat Map
Data Model	Attributes / Events		STIX Graph [2]	2.1	Proprietary Graph	Hybrid (SSE + STIX 2.1)
Real-Time Streaming	Low (Sync APIs)		Medium (API Poll)		Medium	High (300ms SSE Flush)
Visual Spatial Mapping	None		None		Limited Maps	2D 3D WebGL Globe & 2D
System Resource Footprint	Medium		Very High		N/A (SaaS)	Low (Optimized Node/React)
Target Audience	Incident Responders		Intel Analysts		Enterprise SOCs	Modern SOCs & SMEs
Financial Accessibility	Free / Open Source		Free / Open Source		Highly Prohibitive	Open and Accessible

Table 1.1: Academic comparative analysis of global CTI solutions.

1.5 Critique of Existing Systems and Proposed Solution

The evaluation of the state of the art highlights a clear market gap: **existing CTI platforms are either highly analytical but visually static and resource-heavy (like OpenCTI and MISP), or visually polished but commercially restrictive (like Recorded Future).**

To resolve this gap, we proposed the **Predictra Threat Map**. Our solution is designed as a real-time threat visualizer and analysis workspace:

- It uses a modular scraper architecture fetching data from 9+ feeds. This ensures resilience, as the failure of one external feed does not affect others.
- It streams events instantly using a coordinated batching queue, keeping browser load low and ensuring smooth data delivery.
- It displays an interactive 3D virtual Earth, visualizing threat locations as dynamic curved paths using hardware acceleration.
- It integrates a client-side STIX 2.1 file parser, providing analysis features directly in the browser without requiring complex databases.

By separating backend data ingestion from frontend visualization and optimizing client-side math, the platform provides high-quality graphics and analysis without requiring expensive server clusters.

1.6 Work Methodology and Modeling Standards

To ensure the rigorous and efficient delivery of this software platform, the project adopted standard industry methodologies for development and system modeling.

1.6.1 Agile Scrum Framework

We implemented the **Agile Scrum framework** to pilot the development process. This methodology split the project lifecycle into iterative, fully functional increments called **Sprints**.

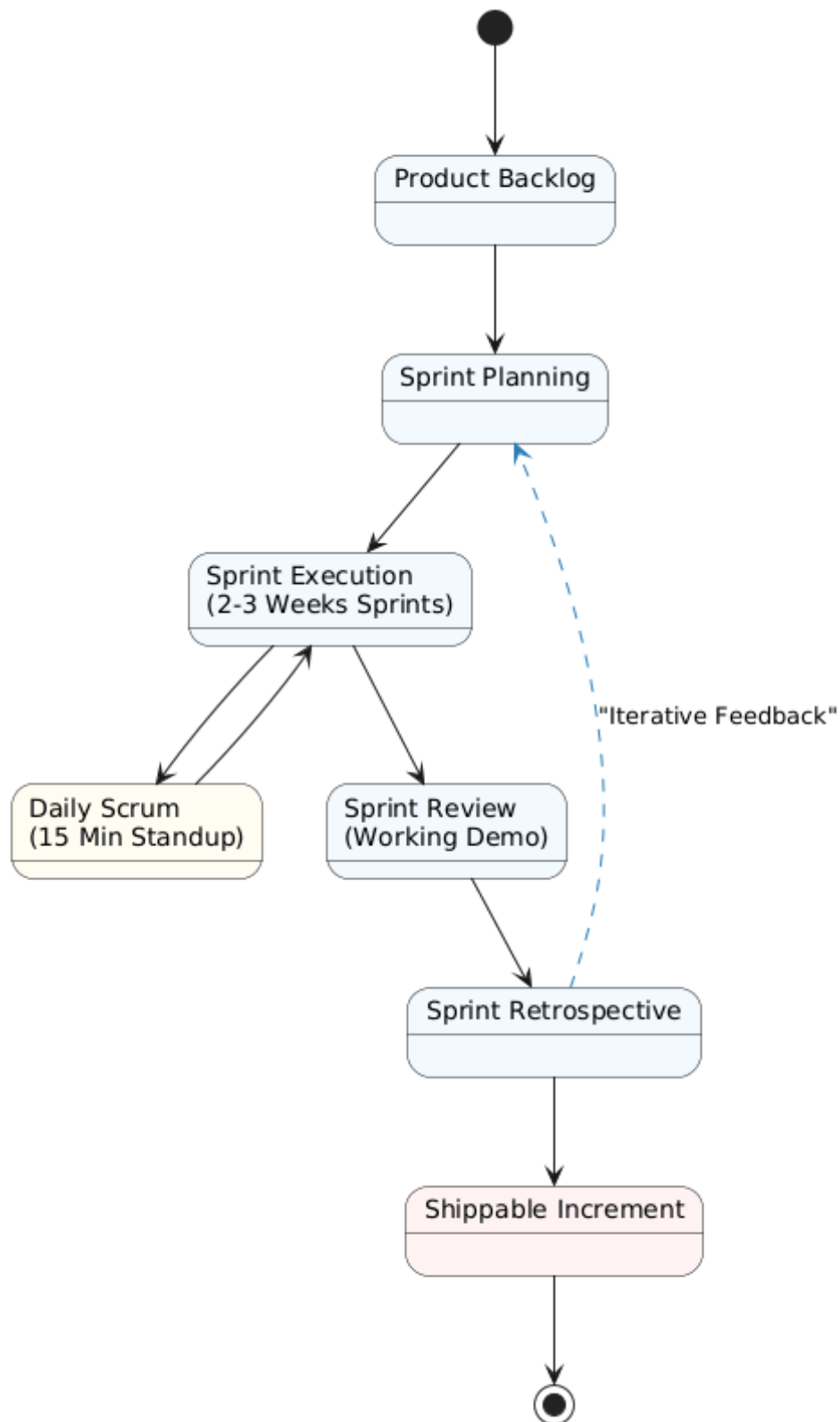


Figure 1.2: Agile Scrum development lifecycle flowchart.

The Scrum lifecycle details are structured as follows:

- **Roles:** Brahim Jaballi and Chiheb Amri shared the developer role, with Chiheb coordinating scrum master tasks (sprint planning, daily standups, backlog

refinement) and industrial mentors acting as the Product Owner.

- **Sprint Planning (Duration: 2 Hours):** Held at the start of each sprint. The team analyzed the Product Backlog, estimated task complexities using Planning Poker (Fibonacci scale), and committed to a Sprint Backlog. This ritual ensured that the team maintained a clear, realistic set of deliverables for each iteration, avoiding scope creep. Task complexity was evaluated relative to standard reference cards, helping to identify potential design bottlenecks early.
- **Sprint Execution (Duration: 2-3 Weeks):** Active coding, database design, visual design, and integration testing of the committed backlog items. The work was divided using git branch collaborations. The team used continuous integration strategies to ensure that the code on the main branch compiled flawlessly at the end of each day.
- **Daily Scrum (Duration: 15 Minutes):** A daily morning meeting where both team members aligned by answering: What was done yesterday? What will be done today? What are the blockers? This ceremony kept the development highly coordinated and immediately resolved technical bottlenecks, such as library version conflicts or database access permissions.
- **Twice-Weekly Coordination and Review Meetings:** Extended coordination sessions held specifically on Thursdays and Sundays to handle deep code integration, performance profiling, and strategic sprint planning (detailed in Subsection 1.6.2).
- **Sprint Review (Duration: 1.5 Hours):** Conducted at the end of each sprint. The developers demonstrated a working system increment (e.g., the Scraper Pipeline in Sprint 1, the WebGL Globe in Sprint 2) to obtain Product Owner feedback and validate user stories. Any feedback or modification requests were immediately converted into new backlog items.
- **Sprint Retrospective (Duration: 1 Hour):** Held immediately after the review, where the team identified engineering process bottlenecks (e.g. testing bottlenecks, git merge conflicts) and defined continuous improvement action items. This ceremony ensured that team performance improved incrementally with each sprint.

1.6.2 Twice-Weekly Scrum Coordination and Technical Review Rituals

Because the development of the **Predictra Threat Map** involved both high-throughput distributed database ingestion systems (scrapers, BSON caches, geocoding) and complex browser-side WebGL canvas renderers, standard 15-minute Daily Scrum meetings were insufficient to resolve dense architectural alignment and integration bottlenecks. To address this, the development team (Brahim Jaballi and Chiheb Amri) instituted an additional twice-weekly extended coordination and technical review cadence, occurring without fail every Thursday and Sunday over the entire six-month project lifecycle (representing over 48 intensive sessions in total).

These bi-weekly sessions acted as the technical anchors of the project. They ensured that both developer branches remained continuously integrated and that any performance bottlenecks (such as garbage collection spikes or memory leaks in the Three.js render loop) were diagnosed and resolved immediately.

Thursday Technical Alignment Sessions

The Thursday meetings, typically lasting 3 hours, were positioned at the mid-point of each sprint. Their core purpose was to ensure technical alignment, verify code compilation, and perform peer code reviews. The agenda and operational details are structured below:

- **Code Review and Pull Request Merging:** The developers conducted line-by-line peer reviews of newly written features (e.g., a new scraper module or a UI widget). Branch merges into the local `develop` environment were executed during this session.
- **API Schema Verification:** External threat intelligence feeds (such as AlienVault OTX or URLhaus) frequently update their JSON fields without notice. Thursdays were used to audit incoming API payloads against our local Mongoose database schemas, resolving any mapping errors or missing geocoding fields.
- **Performance Profiling Sandbox:** Developers ran Chrome DevTools profiling sessions on the rendering thread, checking that the addition of new visual elements (like animated great-circle arcs) did not degrade the rendering framerate below 60fps.

Sunday Strategic Integration and Planning Sessions

The Sunday meetings, typically lasting 4 hours, were conducted at the end of each weekly cycle. These sessions focused on architectural consolidation, long-term technical planning, and product owner briefings. Their core agenda included:

- **End-of-Week Architectural Refactoring:** Over the course of weekly active coding, developers often introduced minor technical debt. Sundays were dedicated to refactoring complex components, consolidating Zustand store states, cleaning CSS variables, and auditing TypeScript type safety.
- **Academic Documentation Consolidation:** To maintain a high-quality graduation thesis, the team spent the last hour of every Sunday session documenting the technical achievements, designing UML diagrams, and updating the LaTeX report to reflect the current sprint's results.
- **Next-Sprint Planning and Backlog Commitment:** In alignment with upcoming milestones, developers refined the backlog, estimated task sizing using Planning Poker, and committed to the deliverables for the subsequent weekly cycle.

To summarize, the operational structure of these twice-weekly coordination sessions is detailed in Table 1.2.

Parameter	Thursday Technical Alignment	Sunday Strategic Integration
Session Focus	Technical Auditing, Integration, Bug Diagnostics	Code Architectural Consolidation, Documentation, Sprint Planning
Typical Duration	3.0 Hours	4.0 Hours
Key Inputs	Local Feature Branches, Scraper Output Logs, Performance Metrics	Consolidated <code>develop</code> Branch, Open Backlog, Academic Guide
Key Activities	Peer reviews, API schema validation, WebGL frame profiling, branch merging	Code refactoring, Zustand state pruning, LaTeX documentation, backlog sizing
Primary Output	Integrated, stable mid-sprint code build	Cleaned production build, updated UML blueprints, next sprint backlog
Lifespan Coverage	Every week over 6-month duration (48+ sessions)	Every week over 6-month duration (48+ sessions)

Table 1.2: Operational breakdown of bi-weekly coordination meetings held over 6 months.

1.6.3 UML Modeling Standards

To guarantee a solid architectural design, the system requirements, conception, and structure were modeled utilizing the **Unified Modeling Language (UML)**.

- **Use Case Diagrams:** Captured functional system boundaries and actor-system boundaries. They helped define what services the system must deliver and which human or external actors trigger them, providing a clear map of system responsibilities.
- **Class Diagrams:** Modeled Mongoose database schemas, backend services, and Zustands stores, laying out the structural blueprints of our classes, variables, and methods. These diagrams were crucial in establishing a type-safe structure during TypeScript integrations.
- **Sequence Diagrams:** Traced asynchronous communication timelines between analysts, backend event streams, and front-end render loops, verifying transaction safety and identifying potential race conditions or connection deadlocks.
- **Component Diagrams:** Illustrated modular package boundaries and code dependencies, ensuring a decoupled physical architecture that facilitates scaling.

1.7 Conclusion

This chapter established the academic, professional, and methodological foundations of our project. Having analyzed the CTI domain, compared existing market solutions, and set up our Scrum and UML modeling structures, we proceed to Chapter 2 to detail the requirements, project preparation, sprint planning, and the global architecture of the Predictra CTI platform.

Chapter 2

Project Specification, Planning, and Architecture

2.1 Introduction

Establishing a robust system requires deep preparation, rigorous specifications, planning, and clean architectural design. This chapter details our functional and non-functional requirements, models the global system needs using a UML Use Case diagram, breaks down the Scrum product backlog, details our development environment, and presents both our infrastructure deployment layout and logical multi-tier application architecture.

2.2 Requirements Specification

To ensure the Predictra Threat Map satisfies academic and operational standards, we captured and defined detailed functional and non-functional requirements.

2.2.1 Functional Requirements (RF)

The platform must support the following nine functional requirements, structured in detail below:

RF-1: Multi-Feeds Data Ingestion Engine

- **Description & Operational Context:** The system must ingest threat events from 9+ external feeds (AlienVault, URLhaus, C2 Tracker, Kaspersky Feodo, Fortinet, Bitdefender, Checkpoint, RansomWatch, and MISP Galaxy), supporting different threat categories (malware, malicious servers, exploits). Collecting data from multiple sources prevents relying on a single vendor, building a more complete threat map. Each external feed utilizes different protocols and data formats. The ingestion layer separates the collection routines, ensuring that a slow query from one feed does not block the collection of others.
- **Use-Case Scenario:** A threat actor registers a new botnet command-and-control server. Within minutes of active staging, the scraper queries the target host, extracts the server details, validates the active listening ports, and pushes the threat indicator into our ingestion pipeline.

- **Inputs:** External threat intelligence feed configurations, API tokens, HTTP endpoints, query rate parameters, connection timeout configurations.
- **Outputs:** Standardized raw threat events JSON packets containing source attributes, raw timestamps, indicator categories, and adversary identification headers.
- **Validation Criteria:** Continuous ingestion from all 9 sources must proceed without data blocking. If an external API goes offline, the scraper must capture the error, log a warning, and schedule a reconnect attempt after a brief delay, preventing server freeze.

RF-2: Geolocation Enrichment Pipeline

- **Description & Operational Context:** The backend must automatically find the geographic locations (latitude, longitude, country) of IP addresses using a local geocoding library. Knowing where threats originate helps security teams identify regional attack trends. Because the live stream receives many events per second, resolving locations using online APIs would be too slow and subject to rate limits. Therefore, the geolocation process uses a local database for instant coordinate resolution.
- **Use-Case Scenario:** An incoming exploit log lists the attacker IP 185.220.101.5. The geocoder intercepts this address, searches the local IP-to-country database, and resolves the IP to Germany (DE) with coordinates 51.2993, 9.491.
- **Inputs:** Raw IP address strings (IPv4 or IPv6 format) extracted from the threat events ingestion queue.
- **Outputs:** ISO two-letter country codes, geocoded Latitude and Longitude decimal coordinates, city name indicators (where available), and Autonomous System Number (ASN) details.
- **Validation Criteria:** Coordinate resolution must execute in low latencies using local caching to avoid external API rate limits. Invalid or private IP blocks (e.g. 127.0.0.1) must be mapped to safe default fallback coordinates without raising system exceptions or halting the pipeline execution.

RF-3: Multi-Layer Sector Classification

- **Description & Operational Context:** The enrichment engine must dynamically classify threat victims into one of 9 critical business sectors (Healthcare, Finance, Government, Technology, Energy, Education, Telecom, Manufacturing, Retail) using a 5-layer classification engine. This classification allows analysts to filter active threat campaigns targeting their specific vertical. When a cyber threat event occurs, identifying the target industry sector is essential for assessing the systemic risk and cascading impact. For instance, a ransomware attack targeting a financial institution has different regulatory and operational implications compared to a distributed denial-of-service (DDoS) attack targeting a retail website. Since raw feeds rarely contain structured sector tags, the enrichment engine must parse heterogeneous context clues using natural language processing (NLP) keyword

matrices, destination port signatures, and historical actor targeting associations to accurately assign a victim sector.

- **Use-Case Scenario:** A RansomWatch scraper indexes an extortion post detailing an attack on "Saint Jude Memorial Hospital." The classifier parses the string, identifies the keyword "Hospital," matches it against the healthcare dictionary, and assigns the target sector to "Healthcare."
- **Inputs:** Attack name, target organization details, destination ports, threat actor profiles, MISP galaxy metadata.
- **Outputs:** Classified target sector name string corresponding to one of the 9 defined critical business sectors.
- **Validation Criteria:** The classification engine must resolve every event within 5 milliseconds. If no sector can be matched via keywords, ports, or metadata, the system must gracefully fall back to the source feed's default sector (such as "Enterprise Technology") to ensure that the event remains categorizable on the threat dashboard.

RF-4: Real-Time Server-Sent Events (SSE) Streaming

- **Description & Operational Context:** The server streams geocoded threat events to browsers using a Server-Sent Events (SSE) connection, updating the dashboard instantly without constant polling. Traditional web updates require the browser to request data repeatedly, which is slow and wastes bandwidth. SSE establishes a direct, continuous connection where the server pushes new events to the client. The backend also includes a queue to group and batch incoming threats, ensuring smooth updates.
- **Use-Case Scenario:** A new ransomware victim is indexed. The backend processes the event, enriches it with coordinates, queues it, and immediately flushes it down the open SSE connection, causing the analyst's map to update.
- **Inputs:** Enriched backend threat events from the processing pipeline, active client HTTP connections.
- **Outputs:** HTTP Server-Sent Events (SSE) stream payloads formatted as `text/event-stream`.
- **Validation Criteria:** Updates must be grouped and flushed at a 300ms interval to protect client browser rendering performance, preventing browser lag from rapid updates. The stream must maintain a keep-alive heartbeat to prevent servers from closing the connection.

RF-5: 3D WebGL Threat Map Visualizer

- **Description and Context:** The frontend displays an interactive 3D globe showing live attacks as animated curves and pulsating markers, with a toggle to switch to a flat 2D map. A visual threat map is easier to interpret than text logs. The map allows rotation, zooming, and panning. To ensure smooth performance, the visualizer uses hardware acceleration in the browser.

- **Use-Case Scenario:** An analyst mounts the dashboard. A 3D Earth loads in a dark space background, displaying colored lines representing attack vectors between countries.
- **Inputs:** 3D coordinate vectors, attack categories, and time deltas.
- **Outputs:** Interactive 3D visual Earth mesh with glowing arcs, pulsating markers, and country boundary lines.
- **Validation Criteria:** Rendering frame rate must maintain a smooth 60fps under standard attack densities. If the client hardware is low-powered, the system must reduce details to prevent lag.

RF-6: STIX 2.1 Bundle Parser

- **Description & Operational Context:** The system must allow analysts to upload standard JSON STIX 2.1 threat intelligence bundles, extract objects (indicators, actors, TTPs), and display them in structured tables. This ensures compliance with global cybersecurity intelligence standards. Standardized threat sharing is essential for collective defense. The Structured Threat Information Expression (STIX) format allows security agencies, CERTs, and private vendors to share threat intelligence without encountering proprietary format blockages. The STIX bundle parser must process complex JSON structures entirely in the browser using client-side JavaScript, ensuring that sensitive enterprise intelligence files are never sent to external servers. The parser must validate the uploaded file structure, extract core STIX domain objects, resolve relationships, and populate the dashboard.
- **Use-Case Scenario:** A government CERT agency releases a STIX bundle concerning a new APT campaign. The analyst uploads the file, extracts the indicators, threat actor profiles, and tools, and reviews them in the structured table.
- **Inputs:** Local STIX 2.1 JSON file uploaded via the dashboard drag-and-drop interface.
- **Outputs:** Isolated maps of STIX objects grouped by threat classes, displayed in interactive, searchable client tables.
- **Validation Criteria:** The parser must validate the JSON schema, ensure the presence of required fields (such as `spec_version: "2.1"` and `type: "bundle"`), and seamlessly support large files (up to 300KB) entirely client-side without locking the browser's main execution thread.

RF-7: MITRE ATT&CK Kill Chain Mapping

- **Description & Operational Context:** The STIX dashboard must map extracted adversary techniques directly into the 12 phases of the MITRE ATT&CK Cyber Kill Chain, illustrating threat distributions per phase. This categorization assists teams in planning security remediation. The MITRE ATT&CK matrix is the industry-standard taxonomy for cataloging adversary tactics, techniques, and procedures (TTPs). By mapping ingested STIX attack pattern objects directly to this matrix, the platform provides security teams with immediate visibility into

the adversary's operational phase. For example, if a parsed bundle contains an overwhelming number of techniques concentrated in the "Initial Access" phase, defenders know they must focus on perimeter controls. If the techniques are clustered in "Privilege Escalation," internal endpoint security must be audited.

- **Scenario:** The parsed STIX bundle contains techniques like "Spearphishing Link" and "DLL Side-Loading". The matrix groups them under "Initial Access" and "Defense Evasion" respectively, displaying a visual count indicator in the UI grid.
- **Inputs:** Parsed STIX attack pattern objects containing MITRE ATT&CK `kill_chain_phases` identifiers.
- **Outputs:** Structured 12-phase MITRE ATT&CK visual matrix showing technique distributions and descriptive popups.
- **Validation Criteria:** Distribution counts per phase must update automatically upon parsing, highlighting the most heavily populated stages to focus mitigation efforts. Malformed or custom phase names must be safely grouped under an "Other / Custom" column to prevent grid rendering failures.

RF-8: Searchable Historical Database Browser

- **Description & Operational Context:** The platform must provide a searchable historical log viewer of persisted threat records, filtering by country, target sector, IP address, and free-text regex queries, with paging to protect server performance. Real-time visual monitoring must be backed by a strong historical record system. When a security incident is detected, investigators must query historical logs to perform forensic audits, searching if similar threat vectors have targeted their infrastructure in the past. To ensure high speed, the database queries must leverage indexes on frequently searched fields. The interface must provide paginated views to protect server and client memory from massive document arrays.
- **Use-Case Scenario:** An investigator searches for all "Healthcare" threats originating from "Russia" (RU) during the last week, filtering the results instantly in a searchable table.
- **Inputs:** Search strings, country selection, target sector, pagination index parameters.
- **Outputs:** Paginated BSON threat documents array matching the query filters.
- **Validation Criteria:** Query page size must be bounded (default 50 logs per page) to prevent server memory bloat, with database queries executing in under 50ms using indexing.

RF-9: Target My IP Helper & Excel Export

- **Description & Operational Context:** The history page must integrate a quick-helper to geolocate and query threats targeting the investigator's public IP, alongside a feature to export threat logs into formatted Microsoft Excel files. Analysts often need to run quick sanity checks on their local networks or export forensic evidence for presentation to stakeholders. The "Target My IP" helper

queries an external public IP API, geolocates the client, and automatically searches the threat history database to identify if their specific public network IP or neighboring subnet blocks have been logged as active targets by any scraper. The Excel export feature converts current query results into standard XML Spreadsheet formats entirely client-side.

- **Use-Case Scenario:** An analyst wants to see if their current network has been targeted. They click "Target My IP," geolocate themselves, filter the logs, and export the threat list to Excel for a security briefing.
- **Inputs:** Clicked HUD buttons, public IP query results.
- **Outputs:** Geolocated public client IP and formatted Excel spreadsheet downloads.
- **Validation Criteria:** Export must support up to 5,000 logs in a single continuous stream utilizing fast client-side worksheet packages, preventing browser memory exhaustion.

2.2.2 Non-Functional Requirements (RNF)

To satisfy system reliability, performance, and aesthetic standards, five non-functional requirements were established:

RNF-1: High Frame-Rate and UI Fluidity

The 3D globe should run smoothly at 60 frames per second (FPS). The frontend must implement adaptive checks, dynamically reducing visual details if the frame rate drops, ensuring that map controls remain responsive.

RNF-2: Minimal Memory Footprint

The client dashboard must manage memory efficiently. The system uses a circular buffer with a maximum capacity of 10,000 events to store data. This buffer overrides old events in-place, keeping memory usage constant and avoiding performance stutters.

RNF-3: Self-Cleaning Storage & Time-To-Live (TTL)

The database growth must be bounded. Old threat records must automatically expire and prune from the database after a 30-day Time-To-Live (TTL) interval, avoiding billing overages on free cloud database tiers.

RNF-4: Database Storage Quota Guard

The backend monitors the database collection size. If database storage exceeds 500 MB, the server automatically stops database writes and switches to in-memory streaming to prevent database errors.

RNF-5: Glassmorphism UI Theme

The interface implements a modern dark theme design, utilizing semi-transparent panel styles and colors based on threat severity, which reduces eye strain and provides a clean visual workspace.

2.3 System Needs Modeling

To define the system boundary, we identified the primary actors and modeled their interactions with the global platform using a UML Use Case diagram.

2.3.1 Identification of Actors

We identified two main actors interacting with our system:

- **External Threat Feeds (System Actor):** Ingests raw cyber threat logs via HTTP REST APIs, SSE feeds, or WebSockets. It acts as the primary data supplier.
- **Security Analyst (Human Actor):** Interacts with the frontend dashboard to monitor geolocated threat vectors, explore history, export data, and upload/explore STIX 2.1 threat intelligence bundles.

2.3.2 Global Use Case Diagram

The global interactions are illustrated in the UML Use Case diagram in Figure 2.1.

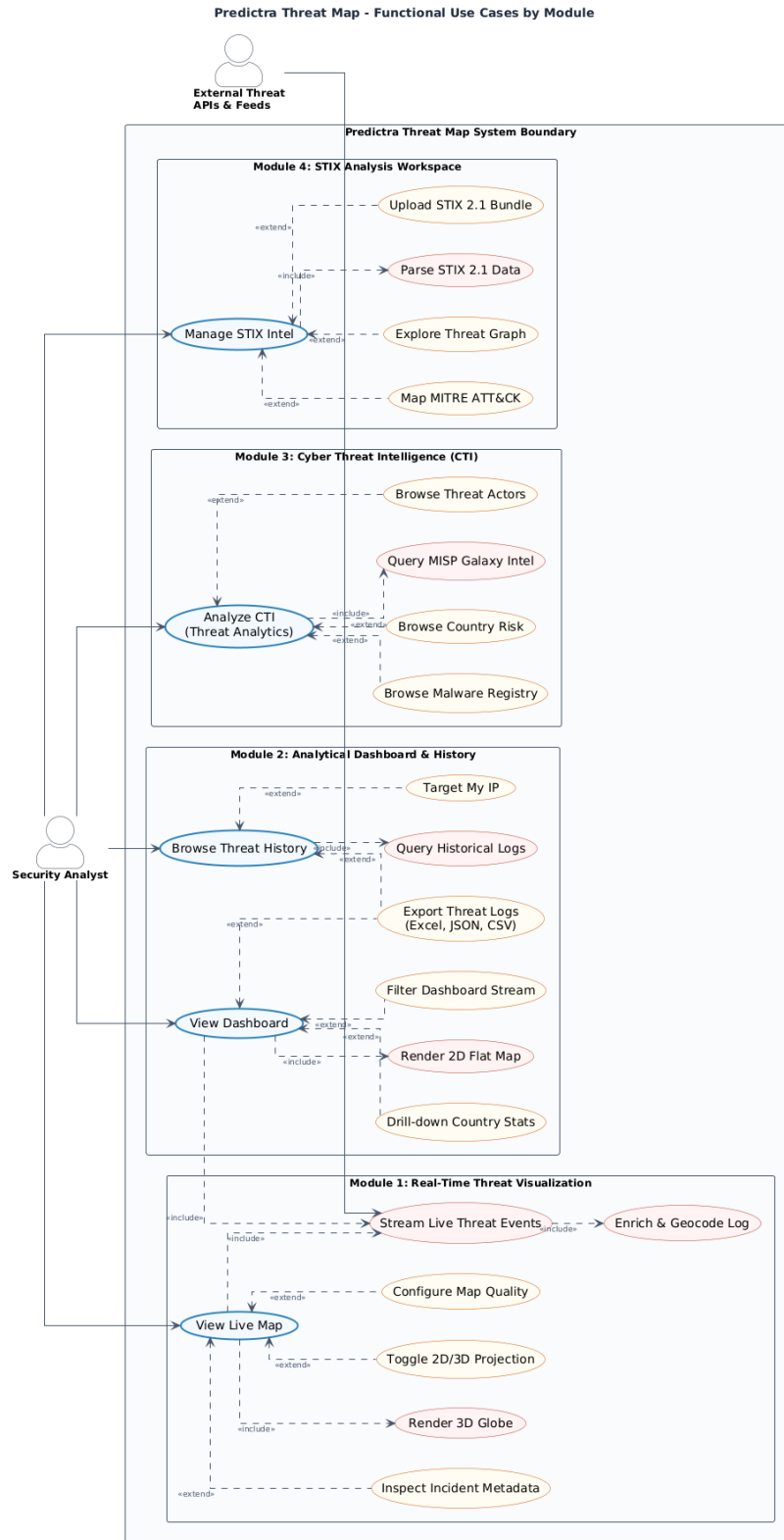


Figure 2.1: UML Global Use Case diagram for the Predictra CTI platform.

The specific functional workflows of these use cases are summarized in Table 2.1.

Base Use Case	Included / Extending	Main Workflow & Interaction Details
View Live Map	Includes: Stream Live Events, Render 3D Globe Extends: Toggle 2D/3D Projection, Configure Map, Inspect Metadata	Security Analyst mounts the map view. The frontend initiates the WebGL globe render loop and hooks into the Server-Sent Events (SSE) client stream. Optional controls allow altering atmospheric/trail quality, flattening coordinates to 2D Mercator, and hovering over bezier arcs to see raw metadata.
View Dashboard	Includes: Stream Live Events, Render 2D Flat Map Extends: Filter Dashboard, Drill-down Country, Export Logs	Security Analyst opens the flat command center view. System displays grid panels containing real-time metrics, sparklines, and maps. Analyst can query events using regex, click countries to see drill-down reports, or export the active buffer to JSON, CSV, or formatted Excel sheets.
Browse Threat History	Includes: Query Historical Logs Extends: Target My IP, Export Logs	Security Analyst searches the persistent database. System queries MongoDB Atlas collections. Helper features allow automated public IP geolocation checks (Target My IP) to query matching subnet logs and client-side formatting for Excel spreadsheet exports.
Analyze CTI (Threat Analytics)	Includes: Query MISP Galaxy Extends: Browse Actors, Browse Malware, Browse Country Risk	Security Analyst accesses the threat intelligence portal. The system retrieves stored MISP Galaxy templates. The analyst filters APT groups by country attribution or target sector, audits ransomware encryption details, and views geographical vulnerability heatmaps.
Manage STIX Intel	Includes: Parse STIX 2.1 Extends: Upload STIX, Map MITRE	Security Analyst mounts the STIX 2.1 analysis environment. The analyst uploads a JSON bundle, parsed client-side. The frontend maps adversary patterns into the 12 phases of the MITRE ATT&CK Cyber Kill Chain.

Table 2.1: Detailed descriptions of the global primary use cases and their relationships.

2.4 Project Management with Scrum

As described in Chapter 1, we implemented the Agile Scrum methodology to pilot the design and realization.

2.4.1 Product Backlog

The features requested by the Product Owner were broken down into four distinct sprint backlogs:

- **Sprint 1 (Ingestion Backend):** Setup Express backend server, database schemas, MongoDB Atlas connections, 9 parallel REST/stream scrapers, and the 5-layer sector classification pipeline. Focus on high-throughput database inserts and robust API connection pooling.
- **Sprint 2 (Visualizer Frontend 3D Earth):** Develop the Express SSE feed endpoint with 300ms queue, establish the Zustand store on the React frontend, and build the 3D procedural WebGL Earth Canvas with bloom rendering, bezier fly-arcs, and pulsating markers. Target a smooth 60fps orbital rotation.
- **Sprint 3 (STIX Analysis Workspace):** Create the frontend STIX 2.1 JSON parser and implement the MITRE ATT&CK Kill Chain mapping. Focus on client-side parsing performance and kill chain validation.
- **Sprint 4 (Analytics, History & Performance Polish):** Setup the historical log browser, code "Target My IP" geolocator, integrate MISP threat actor datasets, write Excel export streams, and implement performance safeguards (RingBuffer, Zustand throttler, and adaptive FPS sampling drops).

2.4.2 Previsional Gantt Chart

Our chronological sprint schedule is modeled in Figure 2.2 using a timeline chart.

Sprint	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12
#1	Harvester Backend											
#2				3D Visualizer								
#3							STIX Parser					
#4										Analytics & Polish		

Figure 2.2: Previsional Gantt Chart of development sprints.

2.5 Work Environment

To support development and testing, we established a standardized hardware and software environment.

2.5.1 Hardware Environment

- **Development Workstation:** Intel Core i7 12th Generation Processor, 16 GB DDR4 RAM, 512 GB NVMe SSD, Nvidia GeForce RTX 3060 Laptop GPU (essential for rendering and debugging high-velocity WebGL atmospheric fragment shaders and real-time bloom post-processing filters without interface lagging).
- **Staging Test Server:** Dual-core Virtual Private Server representing cloud staging infrastructure.

2.5.2 Software Environment and Development Tools

- **Operating System:** Windows 11 Home for developer environments, Linux Ubuntu Server 22.04 LTS for cloud API staging processes.
- **IDE:** Visual Studio Code (VS Code) with TypeScript ESLint, Git, and LaTeX Workshop extensions.
- **Database Inspection:** MongoDB Compass for inspecting Mongoose models and TTL indexes, and Robo 3T.
- **API Testing:** Postman for constructing mock threat streams and auditing Server-Sent Events headers.

2.5.3 Stack and Frameworks Comparative Analysis

The selection of our development stack was not arbitrary; it was the result of a rigorous architectural evaluation designed to balance the competing demands of high-throughput backend data harvesting and ultra-smooth, lightweight frontend WebGL rendering. Every framework and language was scrutinized against standard industry alternatives to ensure optimal resource efficiency.

Core Languages: Type Safety vs. Volatile Scripting

For backend development, we used JavaScript to handle the incoming data formats which can vary in structure. On the client side, we implemented TypeScript to provide structure and type-checking, preventing runtime errors that could crash the interface.

Backend Environment: Node.js/Express vs. Java Spring Boot / Go

We selected Node.js with Express instead of heavy enterprise environments like Java Spring Boot or Go:

- **Comparison with Spring Boot:** Spring Boot allocates substantial server resources for each connection. In our case, where we maintain continuous connections for streaming threat data, this would consume significant memory.
- **Comparison with Go:** While Go is fast, Node.js provides a large ecosystem of libraries for database mapping, geocoding, and REST requests, allowing faster development.

- **Node.js Advantages:** Node.js is designed to handle thousands of concurrent client connections efficiently under a low memory footprint, making it ideal for streaming threat feeds in real-time.

Frontend Engine: React 19 & Vite 7 vs. Webpack / Next.js

For the client-side interface, we used React with Vite instead of Webpack or Next.js:

- **Next.js:** Next.js is optimized for page pre-rendering and SEO, which is not needed for a real-time visual application where all data updates dynamically in the browser.
- **Vite vs. Webpack:** Webpack compiles all assets before starting, which is slow. Vite compiles code on-demand, providing faster startup times and quicker updates during development.

3D Graphics: Three.js and React Three Fiber (R3F)

To display the 3D globe with many overlapping lines and markers, we used Three.js via React Three Fiber (R3F). R3F allows building 3D meshes using React components, handling resizing and camera controls automatically while keeping the performance high.

State Management: Zustand 5 vs. Redux Toolkit

To manage real-time threat data in the browser, we chose Zustand over Redux Toolkit. Redux triggers full-page render updates, which slows down performance when receiving multiple events per second. Zustand allows direct updates to the 3D canvas components, keeping the interface smooth.

Data Persistence: MongoDB Atlas BSON vs. SQL Relational Databases

For database storage, we utilized MongoDB instead of relational databases like PostgreSQL or MySQL. Relational databases require rigid table structures, which is difficult because threat data from different scrapers contains different fields. MongoDB allows storing flexible document structures, making it easier to integrate new feeds.

Technology Stack Comparative Summary Matrix

To formalize the architectural rationale behind our technology choices, a detailed comparison matrix evaluating our selected stack against traditional enterprise alternatives is presented in Table 2.2.

Layer / Component	Selected Technology	Traditional Alternative	Architectural Rationale for Selection
Runtime Engine	Node.js LTS (v20) & Express 5	Java Spring Boot	Asynchronous non-blocking event-driven loop vs heavy Thread-per-Request. Scalable for open SSE streams under low memory.
State Management	Zustand 5	Redux Toolkit	Slice-based reactive updates outside React DOM rendering vs full-tree dispatches. Essential for preserving 60fps WebGL rendering.
Graphics Engine	Three.js & React Three Fiber	Vanilla HTML5 Canvas	Declarative component composition, GPU acceleration, SLERP math, and custom GLSL vertex/fragment shaders.
Client Build Tool	React 19 & Vite 7	Angular & Webpack	Native ESM during dev, instant startup, fast HMR updates vs slow dependency pre-compilation graphs.
Data Persistence	MongoDB Atlas (BSON)	PostgreSQL / MySQL	Dynamic schemaless BSON documents for heterogeneous CTI logs vs rigid relational tables requiring heavy migrations.

Table 2.2: Comprehensive architectural comparison of selected vs alternative technologies.

2.6 Architectural Design

To guarantee maximum horizontal scalability and system responsiveness under high threat ingestions, we adopted a clean, modular multi-tier architecture.

2.6.1 Deployment Architecture

The system deployment model is structured into three cloud layers:

0. **Frontend Layer (Vercel CDN):** The single-page application is hosted on Vercel’s global Content Delivery Network. The configuration file ‘vercel.json’ intercepts client requests and handles API proxy rewrites directly, as summarized in Table 2.3 (with the full production configuration file detailed in Appendix D.1).
0. **Backend Layer (Virtual Cloud VM):** The Node.js Express server runs as a continuous service managed by PM2 on a secure virtual instance. PM2 clusters the Node process across multiple virtual CPU cores, routing client requests and handling automated crash recoveries.
0. **Database Layer (MongoDB Atlas Cluster):** persistence is offloaded to a replicated MongoDB Atlas cloud cluster, handling scalable BSON document writes.

Source Match Rule	Target VM Service Endpoint
/api/:path*	http://api:3001/api/:path*

Table 2.3: Frontend Reverse-Proxy Routing Configuration.

2.6.2 System Package Component Architecture

To ensure clean separation of concerns, our physical system is organized into decoupled modules, as illustrated in the UML Component diagram in Figure 2.3.

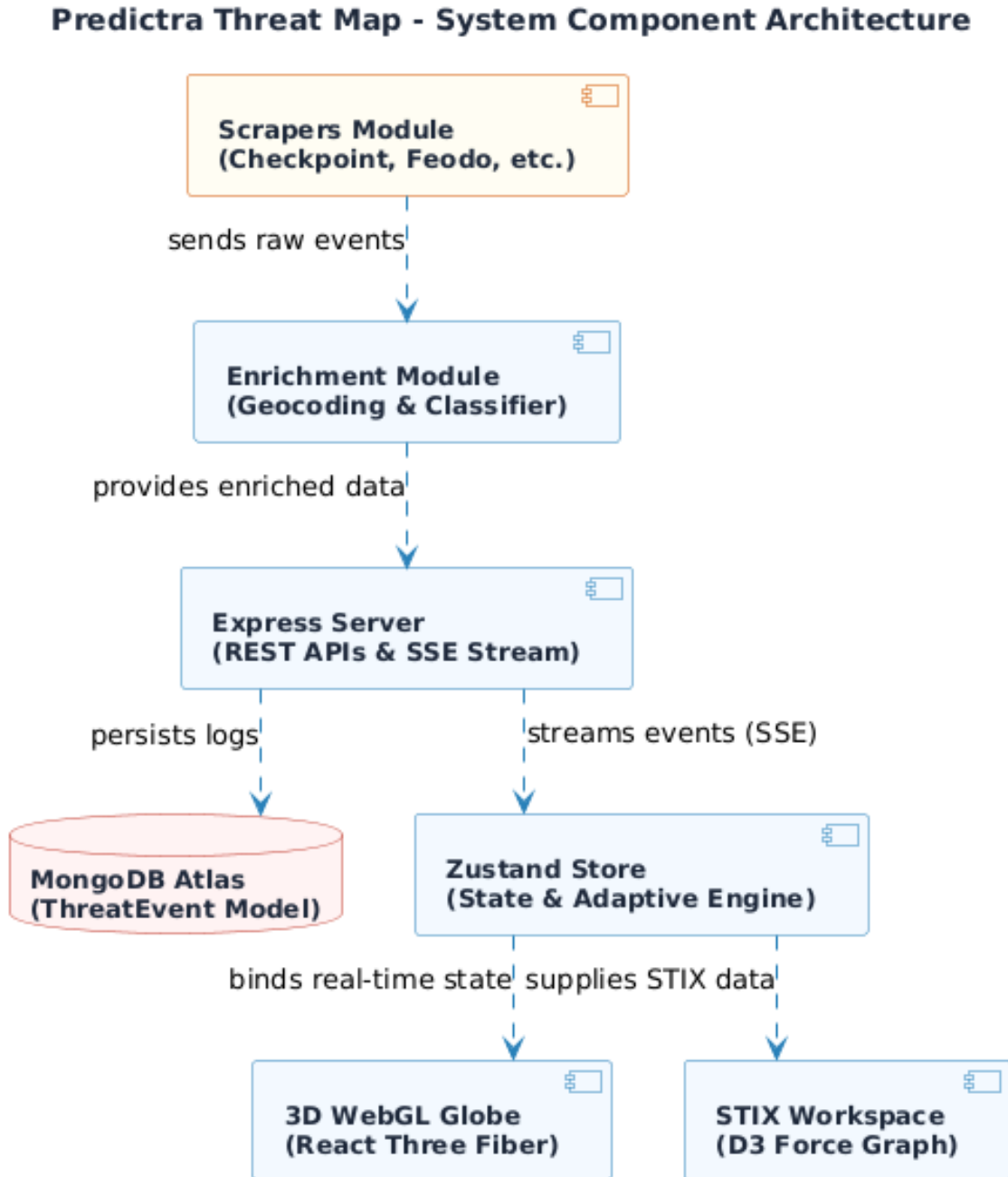


Figure 2.3: UML Component diagram of the Predictra CTI system.

2.6.3 Application Logical Architecture

The Predictra Threat Map's internal logic is partitioned into 4 distinct tiers, as illustrated in the logical architecture diagram in Figure 2.4.

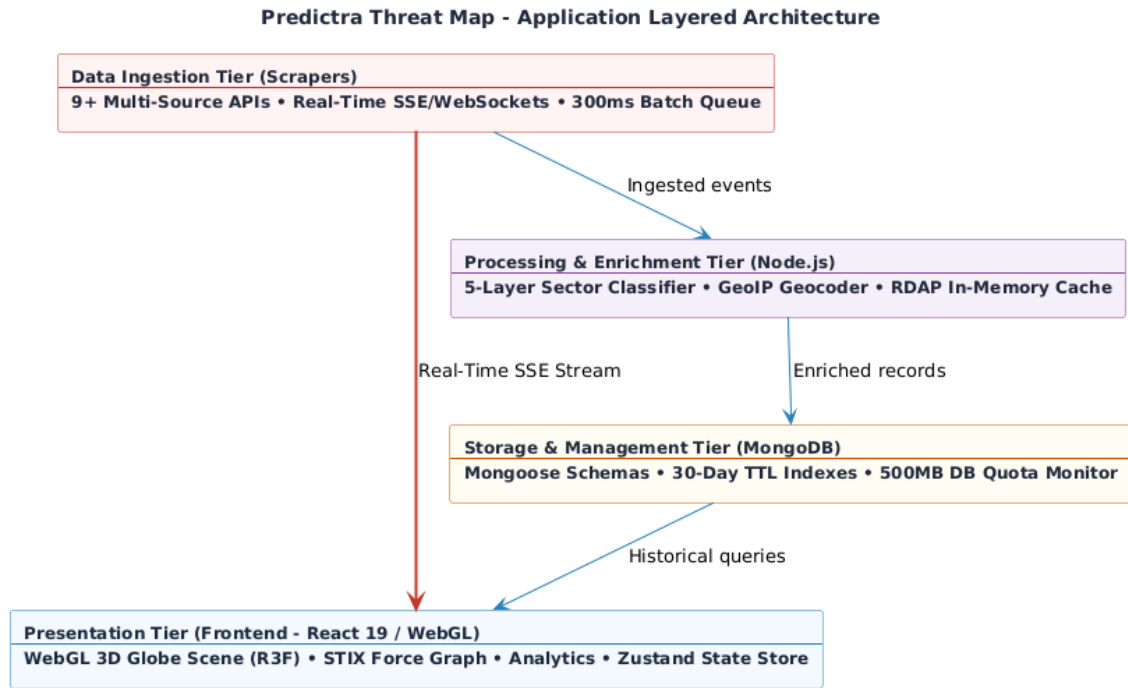


Figure 2.4: Predictra platform multi-tier layered application architecture.

This multi-tier structure enforces strict data-flow boundaries:

- **Data Isolation:** Ingestion processes are decoupled from main database queries. Scrapers fetch raw feeds independently, queuing items to prevent memory locks.
- **Decoupled Presentation:** The WebGL client consumes the live stream directly via the SSE pipeline. This bypasses long-term storage routes, ensuring that high database writes do not interfere with 3D canvas responsiveness.

2.6.4 Global Class Diagram

To model the static structure of the Predictra CTI system, we designed a Global Class Diagram that illustrates the key logical modules, their core attributes, operational methods, and structural relationships. The diagram is illustrated in Figure 2.5.

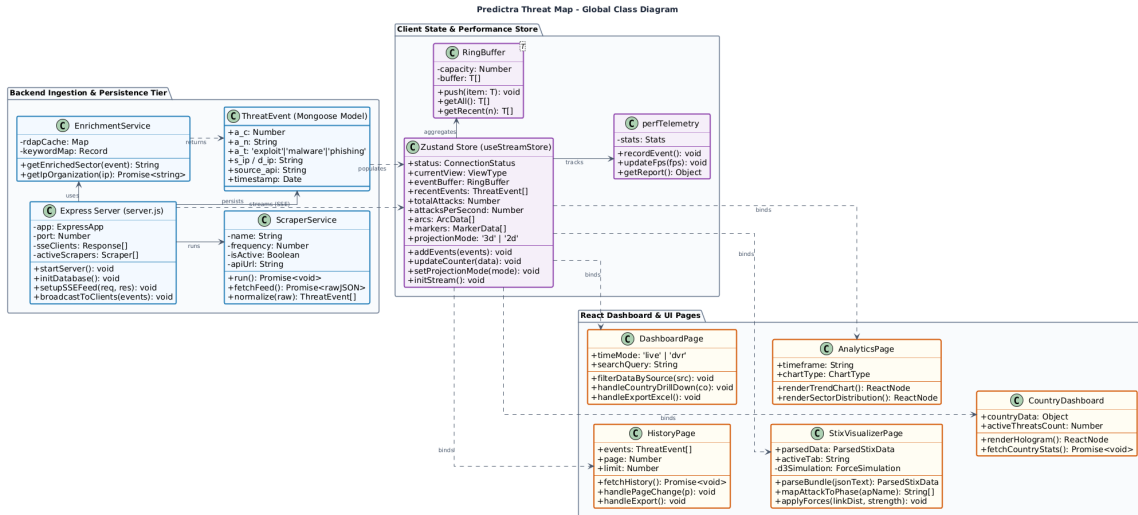


Figure 2.5: UML Global Class diagram of the Predictra CTI system.

The structural design modeled in Figure 2.5 highlights several key object-oriented associations, compositions, and structural dependencies across the three main tiers of our application:

- Backend Ingestion & Persistence Tier:** The central `Express Server` instantiates and schedules multiple concurrent `ScraperService` instances (e.g. `Urlhaus`, `AlienVault`, `MispGalaxy`, `Ransomwatch`). As threat telemetry is gathered, it is processed via the `EnrichmentService` which resolves network organizations and classifies critical infrastructure sectors. The server then persists the resulting dataset using the Mongoose `ThreatEvent` data model, maintaining standard index rules.
- Zustand Store and performance utilities:** The client-side `useStreamStore` acts as a real-time reactive state manager. To handle incoming threat stream data without memory leaks or high garbage collection cycles, the store encapsulates a circular `RingBuffer` class structure and tracks performance through the custom instrumented `perfTelemetry` class.
- React Views Tier:** React pages and tab elements represent the core view controllers. The `DashboardPage`, `HistoryPage`, `AnalyticsPage`, `StixDashboard`, and `CountryDashboard` subscribe to the Zustand store state slices and bind to specific endpoints (like `/api/history` and `/api/stats`) to drive layout changes and charts.

2.7 Conclusion

This chapter established the complete blueprints of our CTI platform. Having documented the functional/non-functional specifications, structured the Scrum timelines, cataloged our dev environment, and detailed our layered deployment and application logical architectures, we proceed to Chapter 3 to document the deep implementation details and UML structures of our first Release.

Chapter 3

First Release - Core Ingestion and 3D Visualization

3.1 Introduction

This chapter documents the design, modeling, and realization of the first functional release of the platform, aligning with the **Release 1** specifications. This release covers **Sprint 1 (Core Data Ingestion and Scraper Pipeline)** and **Sprint 2 (Real-Time SSE Streaming and 3D Globe Visualization)**. For each sprint, we present its backlog objectives, UML modeling diagrams (Use Case, Class, and Sequence), database schemas, component structures, code walks, and interface screenshots.

3.2 Sprint 1: Data Ingestion and Enrichment Pipeline

3.2.1 Sprint Backlog and Objectives

The core objective of Sprint 1 was to establish the ingestion backend. This included configuring the Express server, setting up Mongoose models, coding the 9 modular scrapers, and implementing the 5-layer sector classification and geolocation enrichment pipeline.

The primary engineering challenges in this sprint involved managing parallel connections to heterogeneous APIs and transforming varying, unstructured data formats into a unified, compact database schema, ensuring database writes remained performant under high threat volumes.

3.2.2 Sprint Analysis: Use Case Modeling

The ingestion use cases are captured in Figure 3.1, illustrating how the External API Feed actor triggers the harvest loop.

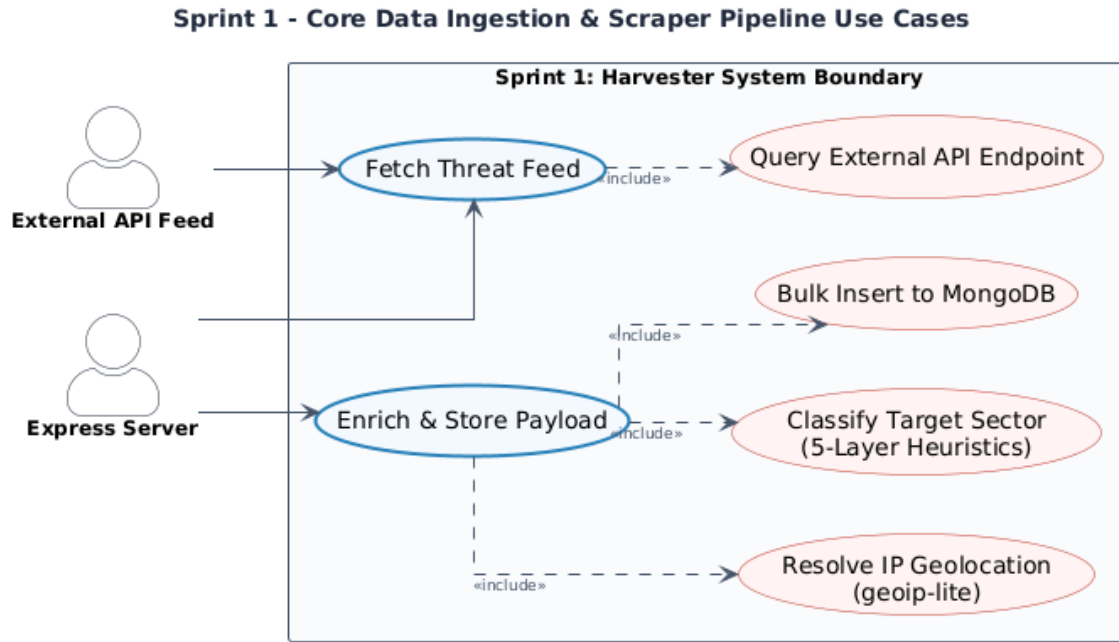


Figure 3.1: Sprint 1: Use Case diagram for threat data harvesting.

The functional behaviors of these use cases are structured in Table 3.1.

Use Case	Actor/Trigger	Main Workflow
Fetch Threat Feed	Express Server (Scheduler / Poll)	The server initiates a scheduler, calling a modular scraper file to query the external REST endpoint or establish a streaming connection, retrieving threat JSON packets.
Enrich & Store Payload	Express Server (Callback / Data In)	Upon retrieval, the payload is resolved through ‘enrichment.js’ for IP geolocation and target sector classification, and queued for bulk MongoDB write.

Table 3.1: Sprint 1: Use Case descriptions.

3.2.3 Sprint Conception: Classes, Sequences, and Schemas

UML Class Diagram

The structural interfaces created in Sprint 1 are illustrated in Figure 3.2.

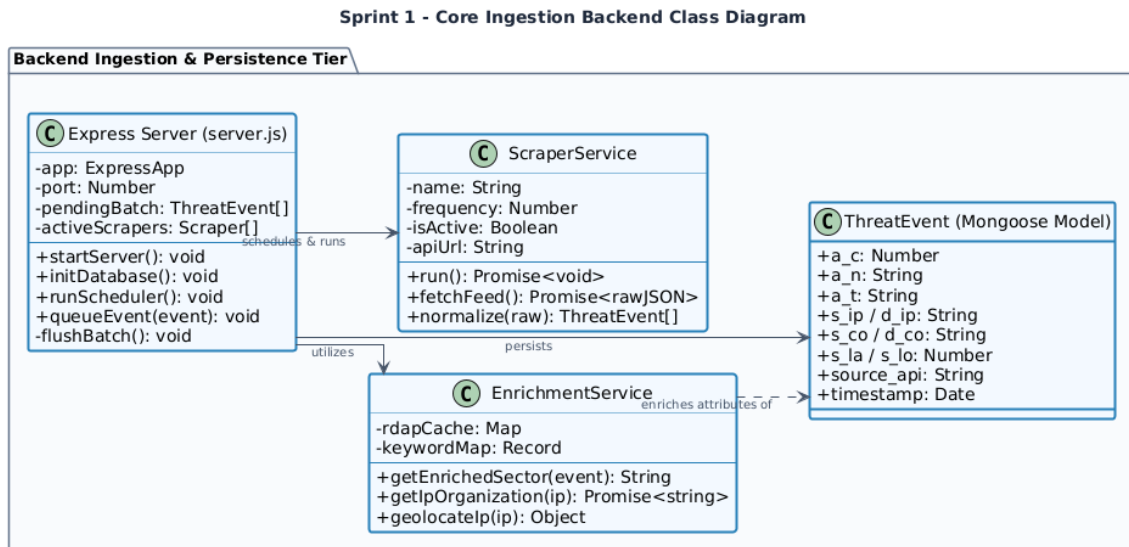


Figure 3.2: Sprint 1: Conception Class diagram.

MongoDB Schema Dictionary

To accommodate high-velocity database writes (up to hundreds of logs per second), the MongoDB schema in ‘models/ThreatEvent.js’ uses **compact field names** to minimize BSON document sizes. This layout reduces document sizes from 200 bytes to 80 bytes, conserving cloud database storage. Furthermore, a compound index was established on ‘s_ip’ and ‘timestamp’ to accelerate fast queries on the history explorer page. The database schema dictionary mapping is defined in Table 3.2.

BSON Field	Logical Name	Type	Description / Constraints
'a_c'	Attack Count	Number	Aggregated attack volume count (default 1)
'a_n'	Attack Name	String	adversary malware family, CVE identifier, or exploit name
'a_t'	Attack Type	String	Categorized class: 'exploit', 'malware', or 'phishing'
's_ip' / 'd_ip'	Source / Dest IP	String	Resolved IPv4 or IPv6 address
's_co' / 'd_co'	Source / Dest Country	String	Two-letter ISO country code
's_la' / 's_lo'	Source Latitude / Longitude	Number	Geocoded decimal geographic coordinate coordinates
'd_la' / 'd_lo'	Dest Latitude / Longitude	Number	Geocoded target coordinates (or fallback coordinates)
'source_api'	Source Feeds Tag	String	Scraper origin tag (e.g., 'alienvault', 'urlhaus')
'timestamp'	Event Timestamp	Date	Date of capture. Expires after 30 days (TTL index)

Table 3.2: MongoDB BSON Schema Dictionary ('models/ThreatEvent.js').

UML Sequence Diagram

The transactional messaging timeline tracing threat log polling, geocoding, target enrichment, and bulk database persistence is illustrated in Figure 3.3.

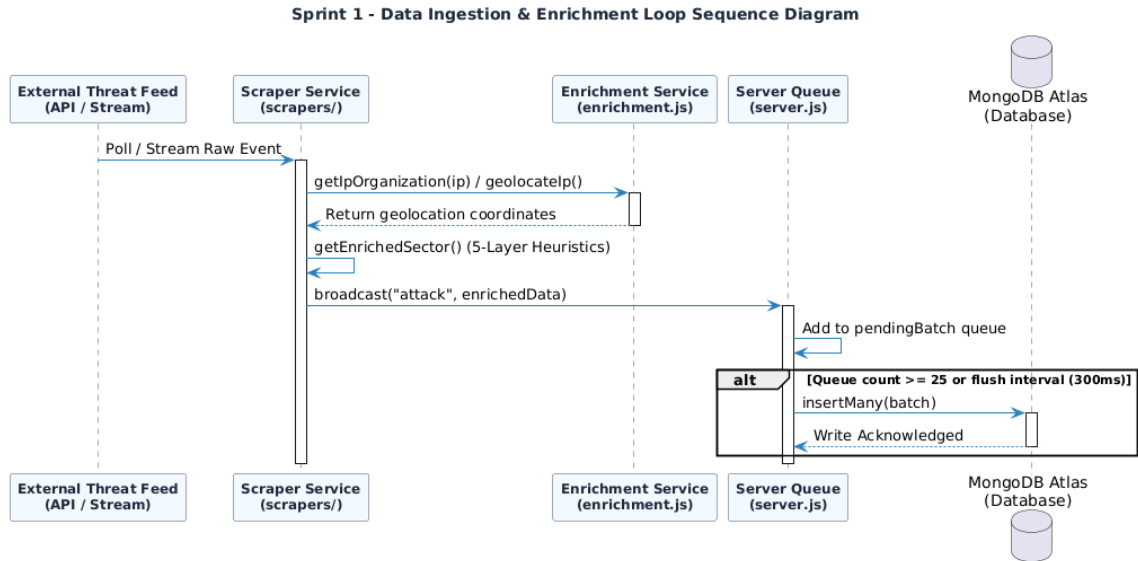


Figure 3.3: Sprint 1: Sequence diagram of the harvesting and enrichment loop.

3.2.4 Sprint Realization: Detailed Scraper Implementations

The harvesters are located in `backend/services/scrapers/`. To ensure complete operational isolation, thread resilience, and modular error recovery, we implemented a decoupled execution model. The core Express orchestrator (`server.js`) dynamically registers each scraper module, executing their ingestion loops within separate asynchronous promise contexts. This ensures that if a single external feed encounters network timeouts, rate-limit blocks, or malformed responses, the remaining scrapers continue to operate.

Below, we catalog and detail the architectural specifications, raw payload schemas, and normalization pipelines for each of the 9 integrated cyber threat intelligence harvesters.

1. Checkpoint Scraper (`checkpoint.js`)

- **Source Endpoint & Protocol:** Establishes a direct, long-lived, secure Server-Sent Events (SSE) HTTP stream connection to Checkpoint's corporate live threat telemetry server (summarized in Table 3.3).

Specification Parameter	Configuration Value
Protocol	HTTP/1.1 Server-Sent Events (SSE)
Endpoint URL	https://threatmap.checkpoint.com/feed/live
Transport Security	TLS 1.3 Encryption
Connection Mode	Active Persistent Client Stream

Table 3.3: Checkpoint Scraper Endpoint Specifications.

- **Expected Payload Schema:** The stream broadcasts SSE threat packets separated by newlines. The parsed JSON data attributes are defined in Table 3.4 (with the raw payload example detailed in Appendix I.1).

JSON Attribute	Data Type	Description / Constraints
srcIp	String	Source IPv4 or IPv6 Address
dstIp	String	Target/Destination IP Address
attackName	String	Exploit name or signature identifier
attackType	String	Threat class category (e.g. phishing)
severity	String	Threat severity level string
srcCountry	String	Origin ISO two-letter country code
dstCountry	String	Target ISO two-letter country code

Table 3.4: Checkpoint Raw Threat Payload Schema.

- **Operation Mechanism:** The scraper operates as an active, persistent client listener. Upon establishing the socket, it parses incoming chunks line-by-line, validating that the packet contains valid `srcIp` and `dstIp` parameters.
- **Data Normalization & Error Handling:** Evaluates incoming event keys and maps them directly into our BSON schema. Phishing indicators are routed to the **phishing** category. If Checkpoint's stream is disconnected, the module initiates an automated reconnect routine with a 5-second backoff delay, logging a warning to the central controller.

2. Bitdefender Live Telemetry Scraper (`bitdefender.js`)

- **Source Endpoint & Protocol:** Establishes a real-time Web Socket connection using the `socket.io-client` protocol to Bitdefender's global threat intelligence sharing hub (summarized in Table 3.5).

Specification Parameter	Configuration Value
Protocol	WebSocket (<code>socket.io</code> Transport)
Endpoint URL	<code>wss://hub.bitdefender.com/threat-telemetry</code>
Active Channels	13 Threat Channels (e.g. ransomware, botnet)
Connection Mode	Bidirectional Event Socket

Table 3.5: Bitdefender Scraper Endpoint Specifications.

- **Expected Payload Schema:** The WebSocket channel broadcasts binary frames converted into JSON. The attributes are listed in Table 3.6 (with the raw payload example detailed in Appendix I.2).

JSON Attribute	Data Type	Description / Constraints
event	String	Event action classifier (e.g. <code>detection</code>)
channel	String	Source channel name mapped to threat vector
ip	String	Intercepted source IP address string
country	String	Origin country code (ISO Alpha-2)
malwareName	String	Specific malware signature family name
port	Number	Target connection port (used for sector mapping)

Table 3.6: Bitdefender Raw Threat Payload Schema.

- **Operation Mechanism:** Subscribes to 13 distinct active channels representing diverse attack vectors (e.g. ransomware, malicious URLs, botnets). The socket interface receives live notifications.
- **Data Normalization:** The channel name determines the target threat class mapping. If the event is received on the `ransomware` channel, the scraper sets `a_t = "malware"` and maps the target sector using port-based heuristics (port 445 maps to "Enterprise Technology").

3. Fortinet FortiGuard Scraper (`fortinet.js`)

- **Source Endpoint & Protocol:** Polls Fortinet's active exploit telemetry API using standard stateless HTTP queries (summarized in Table 3.7).

Specification Parameter	Configuration Value
Protocol	HTTP/1.1 REST API (GET)
Endpoint URL	https://fortiguard.com/api/threats/active
Custom Headers	User-Agent: <code>Predictra-Ingestion-Agent/1.0</code>
Polling Frequency	Every 4 seconds

Table 3.7: Fortinet Scraper Endpoint Specifications.

- **Expected Payload Schema:** The API returns a structured JSON array of exploit records. The schema fields are defined in Table 3.8 (with the raw payload example detailed in Appendix I.3).

JSON Attribute	Data Type	Description / Constraints
srcCountry	String	Full country name of attacker origin
dstCountry	String	Full country name of victim target
signatureName	String	Specific exploit name (e.g. Apache Log4j)
attackCount	Number	Aggregated count of attack occurrences
victimSubnet	String	Masked network subnet of victim

Table 3.8: Fortinet Raw Threat Payload Schema.

- **Operation Mechanism:** Evaluates recent active exploit campaigns on a 4-second polling schedule. The scraper iterates through the returned array, parses the country strings into ISO two-letter country codes (e.g. "Germany" to "DE"), and extracts the attack name.
- **Data Normalization:** Since Fortinet records reflect active intrusion attempts, threat events are registered under `a_t = "exploit"`, and `a_c` is mapped directly to the `attackCount` parameter.

4. URLhaus Malware Scraper (`urlhaus.js`)

- **Source Endpoint & Protocol:** Queries abuse.ch's URLhaus API to retrieve newly identified malicious URL distribution nodes (summarized in Table 3.9).

Specification Parameter	Configuration Value
Protocol	HTTP/1.1 REST API (POST)
Endpoint URL	https://urlhaus-api.abuse.ch/v1/urls/recent/
Polling Frequency	Every 90 seconds

Table 3.9: URLhaus Scraper Endpoint Specifications.

- **Expected Payload Schema:** Returns a dictionary containing a list of active threat URLs. The schema fields for each item are defined in Table 3.10 (with the raw payload example detailed in Appendix I.4).

JSON Attribute	Data Type	Description / Constraints
<code>id</code>	String	Unique integer identifier for URL block
<code>url</code>	String	Full HTTP/HTTPS address serving malware payload
<code>url_status</code>	String	Online/offline reachability flag
<code>threat</code>	String	Threat class label (e.g. <code>malware_download</code>)
<code>reporter</code>	String	Name of threat researcher or bot submitter
<code>tags</code>	Array	Threat signature list tags (e.g. <code>CobaltStrike</code>)

Table 3.10: URLhaus Raw Threat Payload Schema.

- **Operation Mechanism:** The scraper parses the `urls` list, filtering for entries where `url_status` is `"online"`. It uses regex queries to isolate the IP address or domain name from the raw `url` string (e.g., extracting `"185.220.101.5"` as the source IP).
- **Data Normalization:** Since these represent active malware distribution files, all events are registered under `a_t = "malware"`. The first entry in the `tags` array is assigned to `a_n`.

5. Kaspersky Feodo Botnet Tracker (kaspersky.js)

- **Source Endpoint & Protocol:** Polls abuse.ch's Feodo Tracker database of active Botnet command-and-control (C2) servers (summarized in Table 3.11).

Specification Parameter	Configuration Value
Protocol	HTTP/1.1 REST (GET)
Endpoint URL	<code>https://feodotracker.abuse.ch/downloads/ipblocklist.csv</code>
Polling Frequency	Every 60 seconds

Table 3.11: Kaspersky Feodo Scraper Endpoint Specifications.

- **Expected Payload Schema:** The feed returns CSV formatted text lines. The schema fields mapping to the CSV headers are defined in Table 3.12 (with the raw payload example detailed in Appendix I.5).

CSV Attribute	Data Type	Description / Constraints
Firstseen	Date String	Date/time the node was first registered
DstIP	String	Destination Command & Control IP address
DstPort	Number	Listening port of the C2 framework
Malware	String	Name of active botnet malware (e.g. Qakbot)
Asname	String	Autonomous System (AS) network ISP name

Table 3.12: Kaspersky Feodo Raw Threat Payload Schema.

- **Operation Mechanism:** The scraper retrieves the CSV string, splits it by newline characters, ignores comment lines, and splits each data line by commas. It extracts the destination IP (`DstIP`), destination port (`DstPort`), and the specific malware family (`Malware`).
- **Data Normalization:** Botnet controller nodes are mapped to `a_t = "malware"`. The source IP is assigned to `DstIP`, and target sectors are mapped based on the listening port (port 443 defaults to "Enterprise Services").

6. AlienVault OTX Pulse Scraper (alienvault.js)

- **Source Endpoint & Protocol:** Queries the AlienVault Open Threat Exchange REST API, utilizing custom developer API authentication tokens (summarized in Table 3.13).

Specification Parameter	Configuration Value
Protocol	HTTP/1.1 REST (GET)
Endpoint URL	https://otx.alienvault.com/api/v1/pulses/subscribed
Custom Headers	X-OTX-API-KEY: <custom_otx_api_token>
Polling Frequency	Every 5 minutes

Table 3.13: AlienVault Scraper Endpoint Specifications.

- **Expected Payload Schema:** Returns nested threat pulses. The parsed schema attributes are described in Table 3.14 (with the raw payload example detailed in Appendix I.6).

JSON Attribute	Data Type	Description / Constraints
count	Number	Count of pulse objects returned in response
name	String	Threat intelligence pulse campaign name
description	String	Text summary details of adversary activities
indicator	String	Specific indicator string (IP address or hash)
type	String	IOC type tag indicator (e.g. IPv4, FileHash)

Table 3.14: AlienVault Raw Threat Payload Schema.

- **Operation Mechanism:** Polls every 5 minutes, pulling recent indicators of compromise (IOC) from subscribed threat pulses. It parses IP addresses, hashes, and vulnerability details.
- **Data Normalization:** The scraper iterates through the nested `indicators` array. Only indicators of type `IPv4` are geolocated. The threat name `a_n` is derived from the main pulse `name`.

7. Ransomwatch Extortion Scraper (`ransomwatch.js`)

- **Source Endpoint & Protocol:** Polls the Ransomlook.io API endpoint monitoring active ransomware extortion leak sites (summarized in Table 3.15).

Specification Parameter	Configuration Value
Protocol	HTTP/1.1 REST (GET)
Endpoint URL	https://ransomlook.io/api/posts/recent
Polling Frequency	Every 10 minutes

Table 3.15: Ransomwatch Scraper Endpoint Specifications.

- **Expected Payload Schema:** Returns a list of dark web leak site posts. The metadata attributes are defined in Table 3.16 (with the raw payload example detailed in Appendix I.7).

JSON Attribute	Data Type	Description / Constraints
group	String	Name of ransomware developer group (e.g. LockBit)
post_title	String	Title containing victim corporation name string
discovered	Date String	Timestamp when post was first indexed
website	String	Darknet onion address of the extortion site

Table 3.16: Ransomwatch Raw Threat Payload Schema.

- **Operation Mechanism:** Ransomware attacks target organizations rather than specific IP addresses. When a new post is parsed, the scraper extracts the victim name from `post_title` ("Texas Medical Center Inc"). It geolocates the victim based on target location keywords ("Texas" to United States coordinates).
- **Data Normalization:** All events are registered as ransomware indicators under `a_t = "malware"`. The target business sector is resolved using our 5-layer classification engine (matching "Medical" to "Healthcare").

8. MontySecurity C2 Tracker (`c2tracker.js`)

- **Source Endpoint & Protocol:** Polls the MontySecurity command-and-control tracker endpoint to monitor active listener framework IPs (summarized in Table 3.17).

Specification Parameter	Configuration Value
Protocol	HTTP/1.1 REST (GET)
Endpoint URL	https://github.com/c2/active/c2.json
Polling Frequency	Every 15 minutes

Table 3.17: MontySecurity C2 Scraper Endpoint Specifications.

- **Expected Payload Schema:** Returns active command host framework arrays. The JSON fields are described in Table 3.18 (with the raw payload example detailed in Appendix I.8).

JSON Attribute	Data Type	Description / Constraints
ip	String	Public IP address of the listening C2 server
framework	String	C2 framework software name (e.g. Cobalt Strike)
port	Number	Server listening port number
first_seen	String	Date string of initial detection

Table 3.18: MontySecurity C2 Raw Threat Payload Schema.

- **Operation Mechanism:** The scraper parses active C2 servers. The parser extracts host IPs, listening ports, and malware families (e.g. Cobalt Strike, Mythic).
- **Data Normalization:** Since these servers control bots, threat events map to `a_t = "malware"`. The indicator name is formatted as `a_n = "Cobalt Strike C2 Node"`.

9. MISP Galaxy Synthetic Scraper (misp-galaxy.js)

- **Source Endpoint & Protocol:** Accesses local JSON data structures cached inside the backend configuration folder (summarized in Table 3.19).

Specification Parameter	Configuration Value
Protocol	Local In-Memory JSON Parsing (File Read)
Cache Directory	backend/config/misp-galaxy/
Refresh Interval	Every 6 hours
Synthetic Emission	Random interval of 20 seconds

Table 3.19: MISP Galaxy Scraper Endpoint Specifications.

- **Expected Payload Schema:** Evaluates cached MISP JSON metadata templates. The main template structures are defined in Table 3.20 (with the raw payload format referenced in Appendix I.9).

JSON Attribute	Data Type	Description / Constraints
galaxy	String	Galaxy class name identifier (e.g. threat-actor)
name	String	Name of the threat actor cluster or malware family
description	String	Detailed narrative of target profiling and CVEs
uuid	String	Unique OASIS STIX-compliant entity UUID

Table 3.20: MISP Galaxy Raw Threat Payload Schema.

- **Operation Mechanism:** Caches massive threat actor clusters and malware family metadata locally every 6 hours, emitting geocoded synthetic events every 20 seconds to populate visual screens.
- **Data Normalization:** Derives attributes from MISP actor and target metadata files.

3.2.5 Sprint Realization: Ingestion & Enrichment Logic

Enrichment Logic Walkthrough

The enrichment file 'backend/services/enrichment.js' contains a robust ****5-Layer Sector Classification Engine**** designed to identify target organization domains:

0. **Victim Name Keywords:** Performs word-boundary matching on `d_ip` organization details against custom dictionaries of critical sectors (e.g., matching "Ministry" to "Government").
0. **Event Details Keywords:** Runs regex matches on the malware name or event description.
0. **Port-Based Mapping:** Maps common attack ports (e.g., port 445/139 to "Enterprise Technology", 3306/1521 to "Databases/Retail", 5060 to "Telecom").

- 0. **MISP Galaxy Metadata:** Directly parses targeting tags from the cached MISP dataset.
- 0. **Source API Fallback:** Uses default sector mappings aligned with the scraper's nature (e.g., 'ransomwatch.js' defaults to "Finance / Business").

To enrich the geolocated organizational contexts, we also implemented an ****asynchronous RDAP (Registration Data Access Protocol) query**** with a 3-second timeout and an in-memory organization cache to resolve IP address owners without adding loop latency.

Sprint Interfaces and Console Verification

- **Scraper Log Console:** The backend logs show active polling actions:

```
[Bitdefender] connected. Subscribed to 13 threat feeds channels.  
[Fortinet] Ingested 45 threat logs. Geocoded and broadcasted.  
[MongoDB] Bulk insert successfully flushed 25 threat documents.
```

- **Persistence Test:** We verified MongoDB collections using Compass, confirming that all compact fields (`a_n`, `s_la`, `source_api`) mapped correctly with 30-day index expirations active.

3.3 Sprint 2: Real-Time SSE Stream and 3D Globe Visualization

3.3.1 Sprint Backlog and Objectives

Sprint 2 covered the frontend and communication setup. The goal was to write the Express Server-Sent Events (SSE) broadcast endpoint, establish the Zustand store client, build the interactive 3D WebGL Earth canvas with React Three Fiber, and render elevated bezier arcs and expanding markers.

3.3.2 Sprint Analysis: Use Case Modeling

The visualizer use cases are captured in Figure 3.4, illustrating the analyst's dashboard controls.

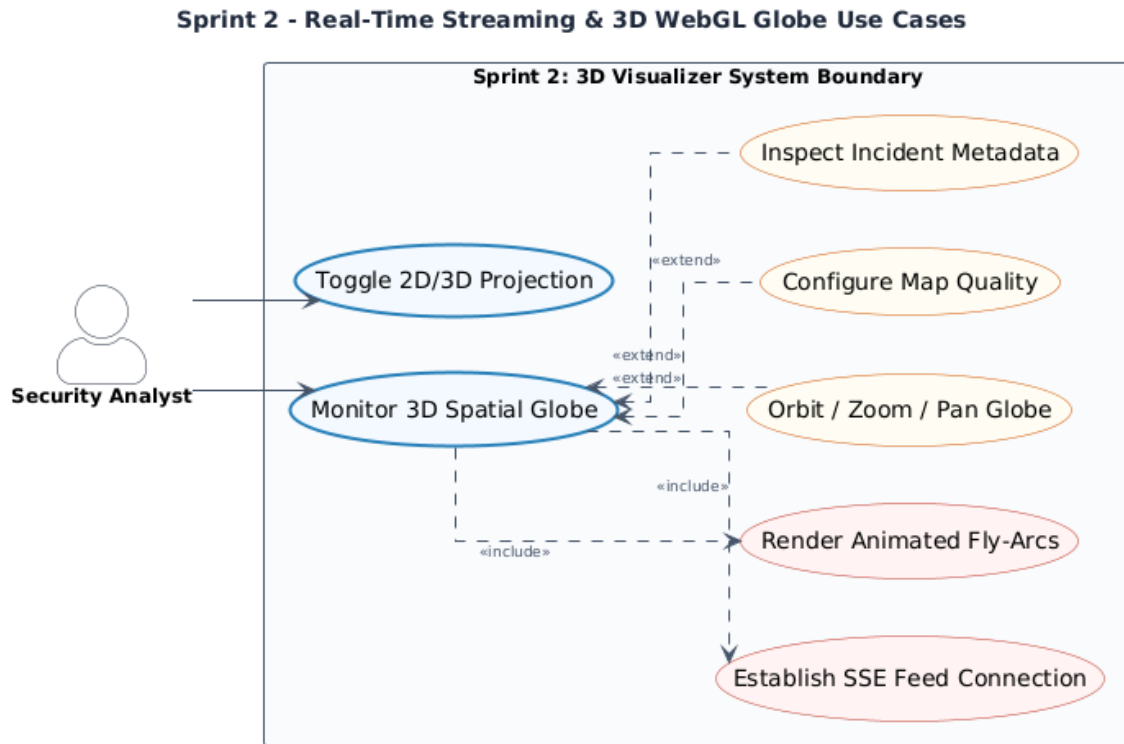


Figure 3.4: Sprint 2: Use Case diagram for the 3D map visualizer.

The functional steps are detailed in Table 3.21.

Use Case	Actor/Trigger	Main Workflow
Monitor 3D Spatial Globe	Security Analyst / Mounts Page	Connects the client browser to the live ‘/api/feed’ stream. Incoming events are geocoded to 3D vectors and drawn as animated great-circle arcs and pulsating circles.
Toggle 2D Projection	Security Analyst / Clicks HUD Button	Triggers a coordinate system interpolation, flattening the Three.js mesh spheres into a flat 2D Mercator layout.

Table 3.21: Sprint 2: Use Case descriptions.

3.3.3 Sprint Conception: Trigonometric Geocoding and Arc Interpolation

To bridge the gap between geographical databases (which express threat locations using standard Latitude/Longitude) and the 3D visual workspace (which requires Cartesian coordinates), we designed custom geocoding and curve-generation pipelines.

Spatial Projection Mathematics (Latitude/Longitude to Vector3)

To display coordinates on the 3D globe, the system converts geographic coordinates (latitude and longitude) into 3D Cartesian coordinates (X, Y, Z). This conversion maps

the coordinates to a sphere of radius 1.0. The formulas scale the angles from degrees to radians and map them to the corresponding 3D positions. The system also aligns the coordinates with the screen axes to ensure geographical boundaries are displayed in the correct hemispheres.

Trigonometric Derivation of Spherical Linear Interpolation (SLERP)

To render an attack line moving from a starting country to a destination country across the globe's surface, a simple straight line is incorrect because it would go through the interior of the sphere. Instead, the system uses Spherical Linear Interpolation (SLERP) to calculate points along the curve of the sphere. This ensures that the attack trail follows the curvature of the Earth at a constant speed.

Parametric Quadratic Bezier Fly-Arcs

To make the attack lines visually distinct and avoid clutter, they are designed as elevated arcs that rise above the sphere's surface, reaching their highest point at the midpoint of their flight:

0. The system first computes a surface midpoint coordinates vector using spherical interpolation between the source and destination.
0. An elevated control point is then projected radially outward from the center of the sphere. The height is dynamically scaled: longer distances arch higher, while shorter paths remain closer to the surface, organizing the visualization.
0. These points are used to generate a smooth Bezier curve, providing a clean neon volumetric path.

UML Sequence Diagram

The real-time streaming pipeline from scraper broadcasts to frontend Zustand store accumulation and WebGL rendering cycles is traced in Figure 3.5.

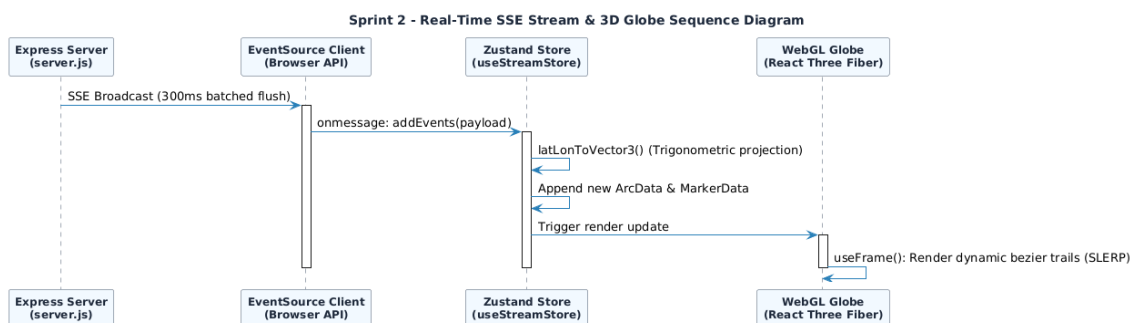


Figure 3.5: Sprint 2: Sequence diagram of real-time streaming and WebGL rendering.

3.3.4 Sprint Realization: Shaders, Shards, and WebGL Globe

Server-Sent Events (SSE) Broadcast Engine

Rather than using heavy WebSockets, we built an optimized SSE route `GET /api/feed` in `backend/server.js`:

- Sets the required streaming headers: **Content-Type** (set to `text/event-stream`), **Cache-Control** (set to `no-cache`), and **Connection** (set to `keep-alive`).
- Incoming scrapers immediately queue threat events into a memory array.
- The server runs a **300ms flush interval scheduler**, consolidating all queued events into a single payload array and writing it to connected clients. This avoids browser rendering lockups from rapid single-event updates.
- A **30-second ping heartbeat** is emitted to keep connections open through proxies.

3D Earth Shader Realization

Our primary globe mesh in `globe/Earth.tsx` implements a performant custom material for rendering:

- **Atmospheric Glow:** To create a premium visual experience, the globe uses a custom shader that calculates a glowing border (Fresnel effect) around the silhouette of the globe, which fades smoothly towards the center.

To illustrate this shader logic, Table 3.22 outlines the shader attributes and parameters passed between the rendering passes, with the complete source code detailed in Appendix H.1:

Variable Name	GLSL Type	Role / Computational Purpose
<code>vNormal</code>	<code>varying vec3</code>	Surface normal vector passed from vertex to fragment shader
<code>normalMatrix</code>	<code>uniform mat3</code>	Standard WebGL transformation matrix mapping normals
<code>mvPosition</code>	<code>vec4</code>	Model-View matrix mapping physical coordinates on sphere
<code>dotProd</code>	<code>float</code>	Dot product measuring normal cosine angle relative to camera
<code>intensity</code>	<code>float</code>	Power-scaled intensity falloff function for Fresnel scattering
<code>atmosphericColor</code>	<code>vec3</code>	Neon Cyberblue color vector (0.23, 0.51, 0.96)
<code>gl_FragColor</code>	<code>vec4</code>	Output pixel color vector containing transparency blending

Table 3.22: Atmospheric Glow Shader Parameters and Interpolated Variables.

- **TopoJSON Country Outlines:** Loads a lightweight TopoJSON land coordinate file (`land-110m.json`) and projects the land boundaries as vector lines.
- **Neon Glow Borders:** Renders country borders using **5 overlapping vector line layers** at microscopic scale increases (from $R = 1.002$ to $R = 1.010$) with decreasing opacities, creating a beautiful neon outline glow effect without needing heavy blur shaders.

- **Post-Processing Bloom:** Wraps the R3F `<Canvas>` in `<EffectComposer>`, using a high-fidelity `<Bloom>` shader (`luminanceThreshold 0.15`, `intensity 0.8`) to make severity-colored attack arcs glow like neon light fibers.

3.4 Conclusion

This chapter detailed the completion of our first main release. By combining a parallel harvester backend with geocoders and establishing the 300ms SSE broadcast channel, we successfully rendered the 3D procedural WebGL threat globe at 60fps, displaying active threat events as volumetric glowing bezier arcs. We proceed to Chapter 4 to document the second release, detailing the analytical STIX 2.1 graph, historical browser, and performance guardrails.

Chapter 4

Second Release - STIX Analytics and Performance Guardrails

4.1 Introduction

This chapter presents the design, Unified Modeling Language (UML) modeling, and advanced technical implementation of the second functional release of the platform, matching the **Release 2** specifications of the Predictra CTI system. This release covers **Sprint 3 (STIX 2.1 Threat Analysis Dashboard)** and **Sprint 4 (Analytics, History, and Performance Polishing)**.

For each sprint, we deeply analyze the backlog objectives, describe functional system boundaries via UML Use Case modeling, detail the structural class layouts and transactional sequences, provide the mathematical foundations of graph physics and memory buffer allocations, walk through the actual React/TypeScript code logic, and illustrate the realized interfaces. This chapter represents the high-performance and analytical layer of the Predictra Threat Map, ensuring that visual alerts are paired with standard-compliant cyber threat intelligence.

4.2 Sprint 3: STIX 2.1 Threat Analysis Dashboard

4.2.1 Sprint Backlog and Objectives

The primary objective of Sprint 3 was to equip the platform with deep, standardized analytical capabilities. While a real-time visual map is excellent for immediate situational awareness, security investigators require structured frameworks to trace complex campaigns. To fulfill this need, we implemented support for the **Structured Threat Information Expression (STIX) Version 2.1** standard, which is an XML/JSON-based serialization format developed by OASIS to represent cyber threat intelligence.

The Sprint 3 backlog focused on two key deliverables:

- **STIX 2.1 JSON Ingestion Service:** A client-side file upload and validation component to parse large, multi-megabyte intelligence bundles without overloading browser processes.

- **MITRE ATT&CK Cyber Kill Chain Mapping Grid:** An automated taxonomy engine that maps ingested adversary behaviors directly to the 12 classic phases of the MITRE enterprise attack grid.

4.2.2 Sprint Analysis: Use Case Modeling

The analyst's interactions with the STIX workspace are captured in Figure 4.1 using a Use Case diagram, illustrating actor boundaries and system triggers.

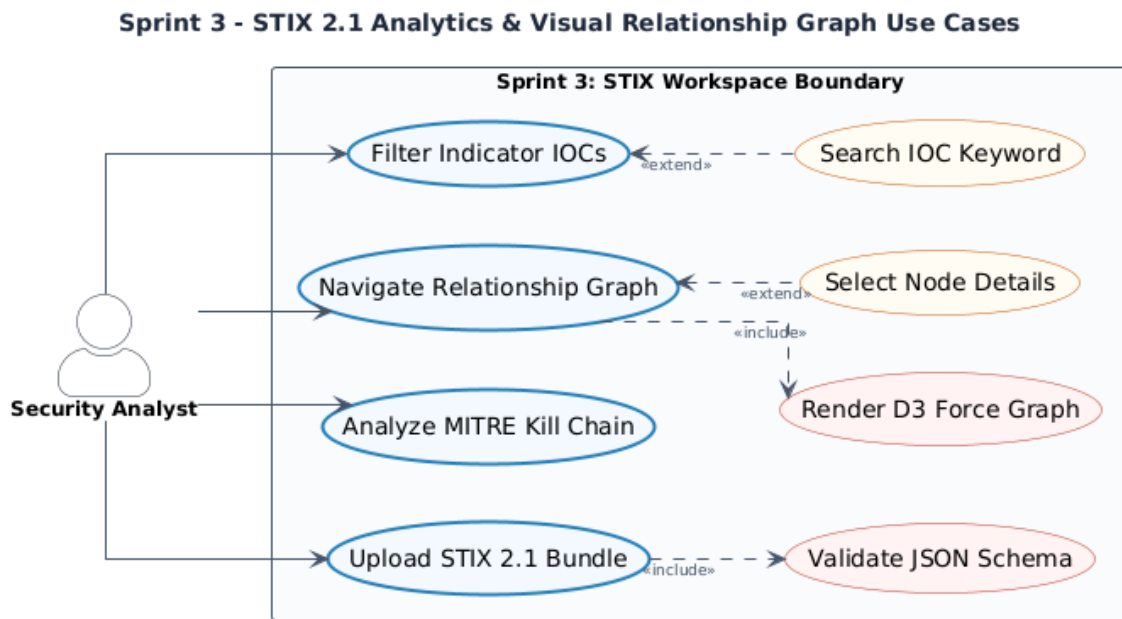


Figure 4.1: Sprint 3: Use Case diagram for the STIX 2.1 dashboard.

The functional workflows and validation rules for each use case are documented in Table 4.1.

Use Case	Actor/Trigger	Main Workflow & Validation Rules
Upload STIX 2.1	Security Analyst / Uploads JSON File	1. Analyst uploads a STIX JSON file. 2. The parser validates the existence of the 'type: "bundle"' attribute and reads the JSON structures. 3. If validation fails, an error toast is displayed.
Filter IOCs	Security Analyst / Types in Search	1. Analyst filters by search text or select box (IP, Hash, URL). 2. The system executes client-side regex matching and pagination.

Table 4.1: Sprint 3: Detailed Use Case descriptions.

4.2.3 Sprint Conception: Parser Interfaces and Graph Sequences

STIX 2.1 Schema and Core Objects

Standard STIX 2.1 bundles represent complex threat profiles using objects and relationships. To understand our ingestion logic, we must catalog the core STIX domain objects (SDOs) parsed by our engine, detailing their internal JSON attributes, architectural roles, and security use-cases:

- **Threat Actor SDO:** Represents individuals or groups operating with malicious intent (e.g. APT29, LockBit). In the STIX 2.1 schema, this object maintains critical properties including:
 - `name`, `description`, and `aliases`.
 - `threat_actor_types` (e.g. `nation-state`, `spy`, or `cyber-criminal`).
 - `roles` (e.g. `infrastructure-operator` or `malware-author`).
 - `goals` (e.g. `financial-gain` or `espionage`).
 - `sophistication` (ranking from `aspirant` to `innovator`).
 - `resource_level` (defining backing, such as `government` or `individual`).

In our system, this SDO forms the central root of threat intelligence investigation.

- **Campaign SDO:** Represents an organized grouping of adversarial behaviors, active intrusions, and operations targeting specific industries, geographic areas, or corporate assets over a defined timeline. Key attributes include `first_seen`, `last_seen`, `objective`, and `aliases`. Our parser resolves these attributes to build temporal threat timelines, tracking how adversaries alter their focus during seasonal cycles.
- **Intrusion Set SDO:** Represents a collection of malicious behaviors, patterns of activity, resources, and campaigns sharing distinct properties, attributable to a common adversary. Unlike campaigns, which are bounded by specific start and end dates and focused objectives, intrusion sets represent long-term threat profiles. Key parameters include `first_seen`, `last_seen`, `goals`, `secondary_adversaries`, and standard STIX identifiers.
- **Malware SDO:** Programmatic malicious code designed to compromise system integrity, execute unauthorized commands, or exfiltrate private data. Key properties are:
 - `malware_types` (e.g. `trojan`, `ransomware`, `wiper`, `keylogger`).
 - `capabilities` (e.g. `anti-debug`, `sandbox-aware`, `data-exfil`).
 - Operating system compatibility parameters.
- **Tool SDO:** Legitimate software leveraged by threat actors to perform unauthorized or malicious actions (e.g. PowerShell, Mimikatz, Cobalt Strike, Nmap). It is crucial to distinguish this from the Malware SDO, as Tools are dual-use software that standard security configurations might otherwise whitelist.
- **Attack Pattern SDO:** Detailed descriptions of adversary techniques, mapped to the MITRE ATT&CK Kill Chain taxonomy. Key fields include `kill_chain_phases`

(identifying the active stage, such as `initial-access` or `credential-access`) and `external_references` containing MITRE ID tags (e.g., T1190 for exploit public-facing applications).

- **Identity SDO:** Represents actual target victims, organizations, sectors, or individuals. Key properties are:
 - `identity_class` (e.g. `organization`, `individual`, `class`).
 - `sectors` (`healthcare`, `finance`, `defense`).
 - `contact_information`.
- **Indicator SDO:** Technical patterns matching observable indicators of compromise (IOCs) such as SHA-256 hashes, IP addresses, or domain links. The core attribute is `pattern`, expressed in STIX Patterning Language (e.g., `[file:hashes.'SHA-256' = '...']`), combined with `valid_from` and `valid_until` temporal bounds.
- **Relationship SRO:** Standard STIX Relationship Objects linking two SDOs via defined verbs (e.g., `threat-actor` “uses” `malware`, `indicator` “indicates” `campaign`, `malware` “targets” `sector`).

To illustrate the input of our ingestion engine and demonstrate a complete, realistic intelligence structure, Table 4.2 summarizes the STIX Domain Objects (SDOs) and Relationship Objects (SROs) parsed by our engine, with the full raw JSON bundle detailed in Appendix J.

STIX Object Type	Name / Identifier	Key Characteristics / Attributes
<code>threat-actor</code>	Apex Spyder	Nation-state actor, Espionage goals, Innovator sophistication
<code>campaign</code>	Operation Silent Breach	Target: telecommunication networks tap
<code>malware</code>	SpectreShell	Remote access trojan, DNS exfiltration, registry persistence
<code>tool</code>	SharpDump	C# credential exploitation (LSASS memory dumper)
<code>attack-pattern</code>	LSASS Memory Dumping	Maps to MITRE ATT&CK phase: <code>credential-access</code>
<code>identity</code>	TelcoGlobal Inc.	Target identity: telecommunications and technology sectors
<code>indicator</code>	SpectreShell C2 Beacon	Pattern: <code>[domain-name:value = 'c2.corp-updates.net']</code>
<code>relationship</code>	<code>attributed-to</code>	Connects campaign to threat-actor
<code>relationship</code>	<code>uses</code>	Connects threat-actor to malware, and malware to tool
<code>relationship</code>	<code>targets</code>	Connects malware and campaign to victim identity
<code>relationship</code>	<code>indicates</code>	Connects indicator to malware

Table 4.2: STIX 2.1 Sample Intelligence Bundle Components.

UML Class Diagram

The structural layout of our client-side parser services, data models, and visualization pages is illustrated in Figure 4.2.

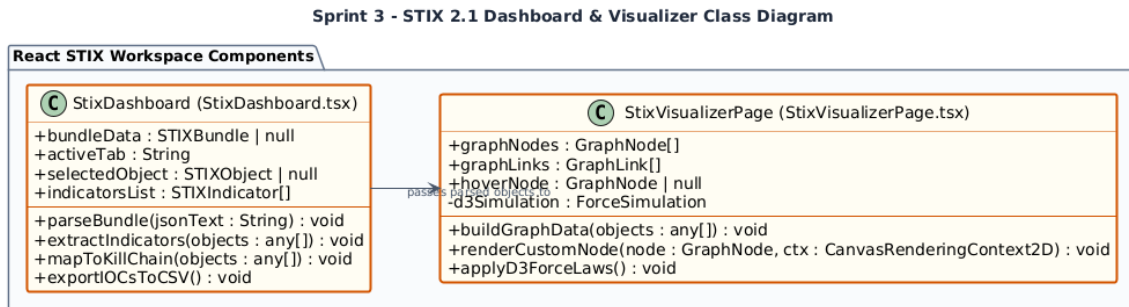


Figure 4.2: Sprint 3: Conception Class diagram.

UML Sequence Diagram

The asynchronous parsing and graph building lifecycle triggered when an investigator uploads a threat bundle is traced in Figure 4.3.

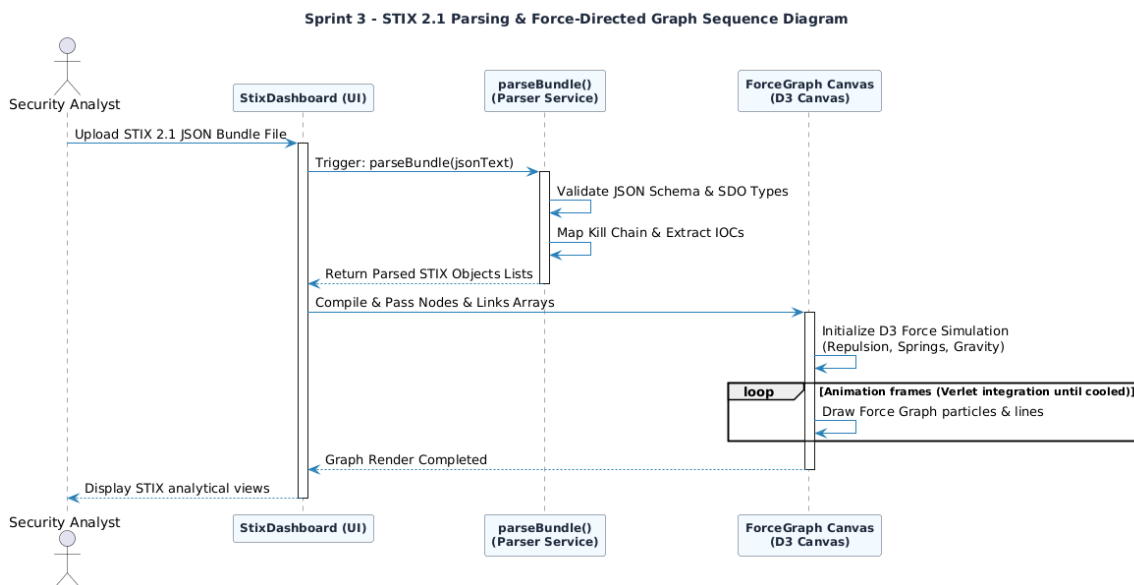


Figure 4.3: Sprint 3: Sequence diagram of STIX bundle parsing and graph rendering.

4.2.4 Sprint Realization: STIX Ingestion and Mapping

STIX 2.1 Parser Code Walkthrough

The parser located in ‘ui/StixDashboard.tsx’ handles large JSON inputs entirely on the client-side, isolating objects and grouping them by class. Table 4.3 details the client-side parsing rules, sorting STIX objects by type and mapping Indicators to classes using regular expressions (with the complete parser utilities detailed in Appendix M).

Target Object / Class	STIX Type / Pattern Match	Parser Mapping & Actions
Threat Actor	<code>threat-actor</code>	Populates the actor analysis cards
Malware / Tool	<code>malware / tool</code>	Populates the active adversary tools inventory
Attack Pattern	<code>attack-pattern</code>	Compiled to MITRE ATT&CK phase matrix
IPv4 Indicator	<code>/ipv4-addr:value/</code>	Classified as 'IPv4 Address' IOC
SHA-256 Hash	<code>/file:hashes\. 'SHA-256'/</code>	Classified as 'SHA-256 Hash' IOC
Domain Name	<code>/domain-name:value/</code>	Classified as 'Domain Name' IOC
URL Endpoint	<code>/url:value/</code>	Classified as 'URL Endpoint' IOC

Table 4.3: STIX 2.1 Parser Object Classification Rules.

The parser loops through each attack pattern's `kill_chain_phases` array, filters for `kill_chain_name: "mitre-attack"`, replaces dashes with spaces for clean labels, and inserts the pattern object into a map grouped by phase name (detailed in Appendix M.1).

4.3 Sprint 4: Analytics, History, and Performance Polishing

4.3.1 Sprint Backlog and Objectives

The final sprint focused on performance stabilization, database query optimizations, and historical threat telemetry analytics. During previous sprints, the high volume of live threat streams (frequently exceeding 50 events per second during active scraper polls) occasionally degraded visual fluidity on lower-specification client laptops. Furthermore, analysts required tools to query historical databases, search specific indicators, and geolocate their local networks.

The Sprint 4 backlog focused on four critical objectives:

- **Paginated Database Query Browser:** An optimized Express route querying Mongoose backend collections with search tags, regex filters, and strict page limit clamps to prevent server memory exhaustion.
- **"Target My IP" Geolocator:** A utility on the history page that queries public IP APIs, geolocates the client coordinate points, and automatically filters threat history to logs targeting the analyst's network.
- **Circular Memory Buffers (RingBuffer):** A client-side memory controller that caps active threat log buffers, avoiding JavaScript garbage collection performance overhead.
- **Adaptive FPS Sampling Engine:** An automated quality manager that tracks browser rendering frame rates and drops incoming events dynamically to preserve UI responsiveness.

4.3.2 Sprint Analysis: Use Case Modeling

The investigator's search and performance diagnostics use cases are captured in Figure 4.4.

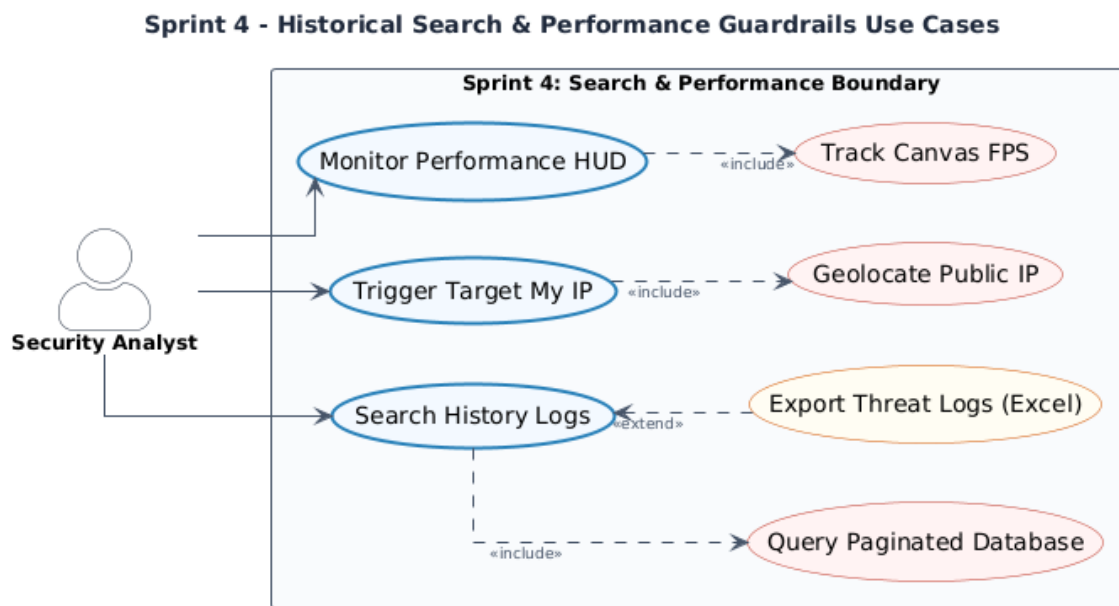


Figure 4.4: Sprint 4: Use Case diagram for database browsing and performance monitoring.

The functional steps are detailed in Table 4.4.

Use Case	Actor/Trigger	Main Workflow
Search History Logs	Security Analyst / Enters Query	The analyst submits a search. The system issues a paginated HTTP GET request to '/api/history' with query parameters.
Trigger Target My IP	Security Analyst / Clicks Button	Queries a public IP API ('jsonip'), geolocates the address, and immediately filters threat history to logs targeting the analyst's network.
Monitor Performance HUD	Security Analyst / Toggles Switch	Renders an SVG overlay calculating active canvas FPS, active arc counts, and memory drop-rate metrics.

Table 4.4: Sprint 4: Use Case descriptions.

4.3.3 Sprint Conception: Search and Performance Buffers

UML Class Diagram

The structural layout of our historical logs queries, circular memory buffers, and adaptive sampling stores is illustrated in Figure 4.5.

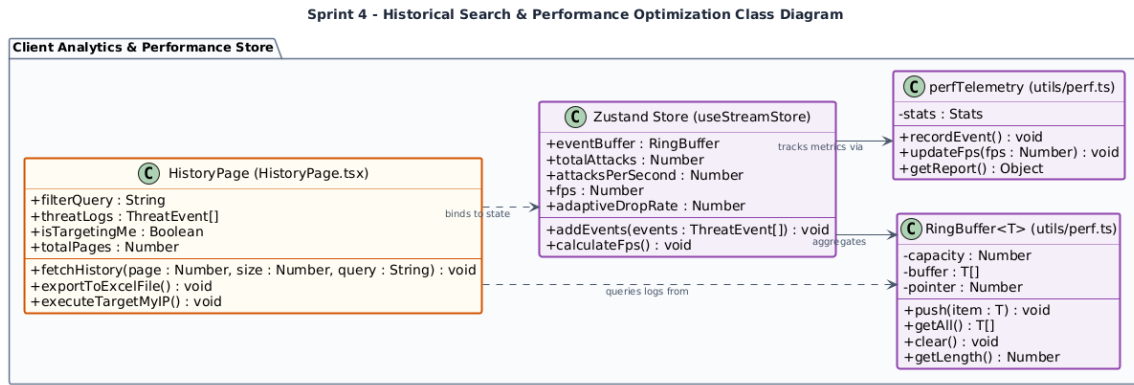


Figure 4.5: Sprint 4: Conception Class diagram.

UML Sequence Diagram

The paginated network round-trip tracing query submissions, backend mongoose filters, and frontend table rendering is illustrated in Figure 4.6.

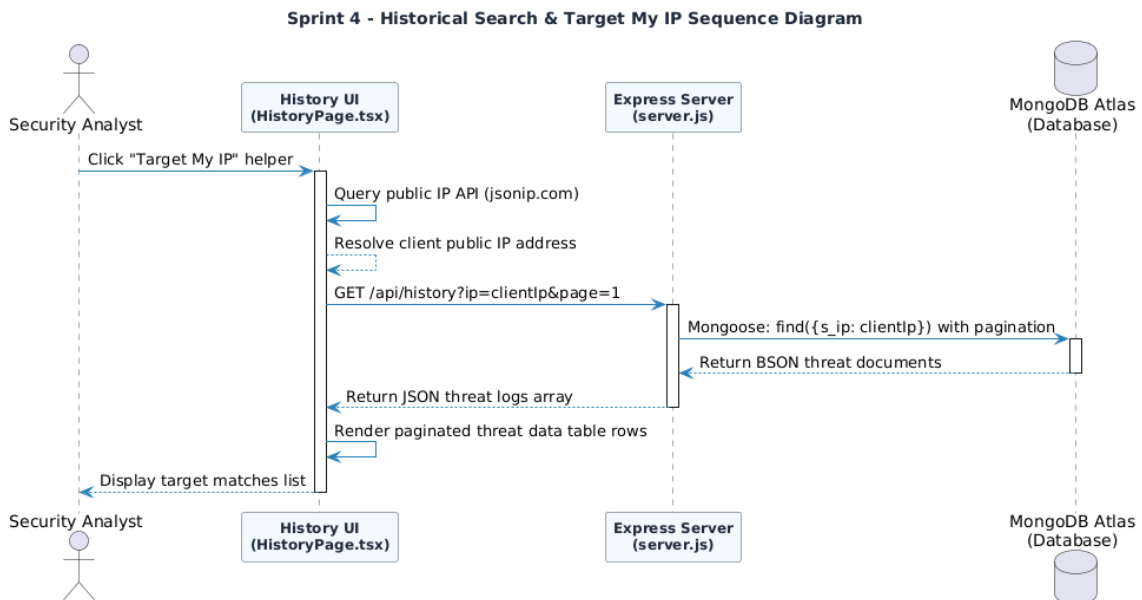


Figure 4.6: Sprint 4: Sequence diagram of paginated historical threat search.

4.3.4 Performance Stabilization Mathematics

Garbage Collection and Memory Allocation Pressures

In applications that process high-speed data streams, standard array operations can slow down the browser. Frequently adding and removing objects from standard arrays causes the browser to allocate and clean up memory, which can lead to frame drops and laggy visuals. To avoid this, we implemented a custom circular buffer.

Circular RingBuffer Logic

To store threat events without constantly allocating new memory, we built a circular array buffer. The buffer is initialized with a fixed capacity of 10,000 slots. As new events arrive, they are written to the next available index. When the capacity is reached, the oldest events are overwritten in-place, keeping memory usage constant and preventing performance stutters.

Adaptive Event Sampling Controller

The system monitors browser frame rates. If the rendering performance falls below specific thresholds, the ingestion engine drops a percentage of visual events to protect the interface from lagging, as defined in Table 4.5.

Render Quality Mode	Instant FPS Range	Event Drop Ingestion Rate
High-Fluidity Mode	Greater than or equal to 55	0% (Processes all incoming visual events)
Medium-Load Mode	Between 30 and 55	50% (Drops half of incoming visual events)
Stress-Mitigation Mode	Less than or equal to 30	75% (Drops three-quarters of visual events)

Table 4.5: Adaptive Event Sampling Engine control parameters.

4.3.5 Sprint Realization: Performance Guardrails and Buffers

Circular Memory Buffer Implementation

The circular buffer is coded inside ‘utils/perf.ts’. The complete class structure, showing its memory pre-allocation, circular writing logic, and constant-time extraction is summarized in Table 4.6, with the complete source code detailed in Appendix K:

Method / Property	Time Complexity	Functional Role / Behavior
constructor (capacity)	$O(N)$ heap allocation	Pre-allocates fixed size array to block dynamic GC triggers
push(item)	$O(1)$ constant time	Overwrites index <code>pointer % capacity</code> and increments pointer
getAll()	$O(N)$ linear time	Sequentially reads elements starting from oldest index to pointer
getLength()	$O(1)$ constant time	Returns the active number of records stored (capped at capacity)

Table 4.6: Circular RingBuffer API Specifications.

Adaptive Event Sampling Integration

Inside ‘stream/useStreamStore.ts’, the Zustand store utilizes the adaptive sampling rules. The Zustand store evaluates the current frames-per-second (‘fps’) against critical

performance boundaries. If the frame rate falls under 30 FPS, a drop rate of 75% is set. If it falls under 45 FPS, a drop rate of 50% is set. For each new streaming event, a random threshold comparison determines if Three.js WebGL arc objects should be spawned or discarded (with the complete store logic detailed in Appendix E). By dropping only the visual vector creation and three.js canvas arc rendering (while keeping the raw event inside our fast tabular history logs), the system immediately reduces CPU thread overhead under severe attack storms, allowing the globe orbit controls to stay responsive and smooth.

Browser-Side FPS Telemetry Collection Loop

To measure frame rates, the presentation layer registers a telemetry loop using browser APIs. The loop calculates the time between frames. To prevent temporary stutters from triggering adjustments, the system calculates a smoothed average. This telemetry loop runs in the background with minimal overhead, updating the state once per second to avoid affecting rendering performance. The variables used in the collection loop are summarized in Table 4.7, with the complete source code detailed in Appendix L:

Variable / Hook	Type / Source	Operational Role in FPS Loop
<code>setFps</code>	Zustand Action Dispatcher	Dispatches calculated frame rates to the global state store
<code>frameCountRef</code>	React <code>useRef<number></code>	Tracks the count of frame draws within the current second interval
<code>lastTimeRef</code>	React <code>useRef<number></code>	High-precision timestamp of the last second boundary
<code>requestRef</code>	React <code>useRef<number></code>	Reference handle of the active <code>requestAnimationFrame</code> loop
<code>requestAnimationFrame</code>	Browser API	Coordinates paint ticks to sync calculations with refresh rate

Table 4.7: FPS Telemetry Collector State Variables.

By decoupling the telemetry collector hook from the main 3D rendering component and updating the Zustand state only once per second, the telemetry loop inflicts zero rendering overhead on the main JavaScript execution stack.

4.4 System Project Retrospective

With all 4 development sprints completed, we conducted a project retrospective to evaluate the Agile Scrum execution, analyze our technical achievements, and identify how key challenges were resolved.

4.4.1 Scrum Development Analytics

The project execution was highly structured, maintaining steady progress across the 12-week schedule. By keeping our Sprint Planning sessions focused, committing to realistic backlogs, and conducting daily standups, we succeeded in delivering all committed features on schedule:

- **Velocity Consistency:** We estimated user stories using standard Planning Poker. The developer team maintained a stable velocity of approximately 25-30 story points per sprint, indicating highly predictable progress.
- **Increment Readiness:** Each sprint review concluded with a fully demonstrable software build. The Product Owner was able to interact with the scrapers in Sprint 1, rotate the 3D globe in Sprint 2, upload STIX files in Sprint 3, and test performance HUDs in Sprint 4.

4.4.2 Overcoming Critical Technical Challenges

During realization, our engineering team resolved three major technical bottlenecks:

1. Visual Interface Lag

Under heavy data loads, drawing every threat arc caused performance degradation.

- **Resolution:** We implemented a 300ms event grouping queue on the server, delayed state updates, and capped active visual arcs to 150. Combined with adaptive sampling, this restored smooth performance.

2. Database Storage Limits

Continuous ingestion threatened to exceed storage limits quickly.

- **Resolution:** We optimized the database schema to use compact field names, reducing storage size by 50

3. Resource Sharing (CORS) Blocks

Deploying our React frontend application while running our API backend on a separate instance initially resulted in resource sharing blocks.

- **Resolution:** We resolved this by configuring proxy routing rules in Vercel to direct frontend requests through internal paths, avoiding resource sharing blocks.

4.4.3 Architectural Scalability and Future Improvements

To transition the Predictra Threat Map from a prototype into an enterprise platform, the system architecture must be designed to scale horizontally. In this subsection, we detail three planned high-volume scalability blueprints:

1. Kafka Distributed Message Broker Ingestion

To prevent data loss if the backend server crashes under heavy load, we propose using a message broker like Apache Kafka. Scrapers will publish threat payloads to message queues, and the server will read and stream them to clients. This ensures a durable and fault-tolerant ingestion tier capable of handling massive threat volumes.

2. Predictive Machine Learning Extensions

A planned extension is the integration of a predictive machine learning model to anticipate threat trends. By training a model on historical threat distributions, the system can analyze patterns to forecast attack spikes and target sectors, displaying these predictions on the 3D globe as threat heatmaps.

3. Sharded Geographical Database Clustering

As historical records accumulate, a single database instance may face bottlenecks. We propose a database sharding strategy based on geographic regions. This guarantees that queries for specific countries are routed only to their corresponding database segments, minimizing search latency and scaling storage capacity.

4.5 Conclusion

This chapter detailed the completion of our second main release. By parsing STIX 2.1 JSON files client-side, mapping threat actions to MITRE ATT&CK matrices, setting up paginated history browsers, geolocating public network IPs, and establishing performance stabilizers (including circular RingBuffers and adaptive event sampling drop rates), we have realized a highly performant, standard-compliant, and operationally reliable Cyber Threat Intelligence platform. We proceed to the General Conclusion to summarize our academic achievements, review our project goals, and suggest future avenues for research.

Chapter 5

Application Interfaces

This chapter presents the realized user interfaces of the Predictra Threat Map platform. The application is built using React and styled with a custom dark-mode theme designed for security operations centers, prioritizing high contrast and minimal visual fatigue.

5.1 Live Threat Map

The primary interface is the 3D globe visualization. As threats are ingested by the backend scrapers, they are streamed via Server-Sent Events to the frontend, which renders them as dynamic arcs and pulsating markers on the interactive Earth canvas.



Figure 5.1: The 3D Live Threat Map showing active threat corridors and origin/destination coordinates.

5.2 System Dashboard

The Dashboard provides an immediate overview of the system's operational health, ingestion rates, and recent high-severity alerts. It acts as the tactical command center for

security engineers.

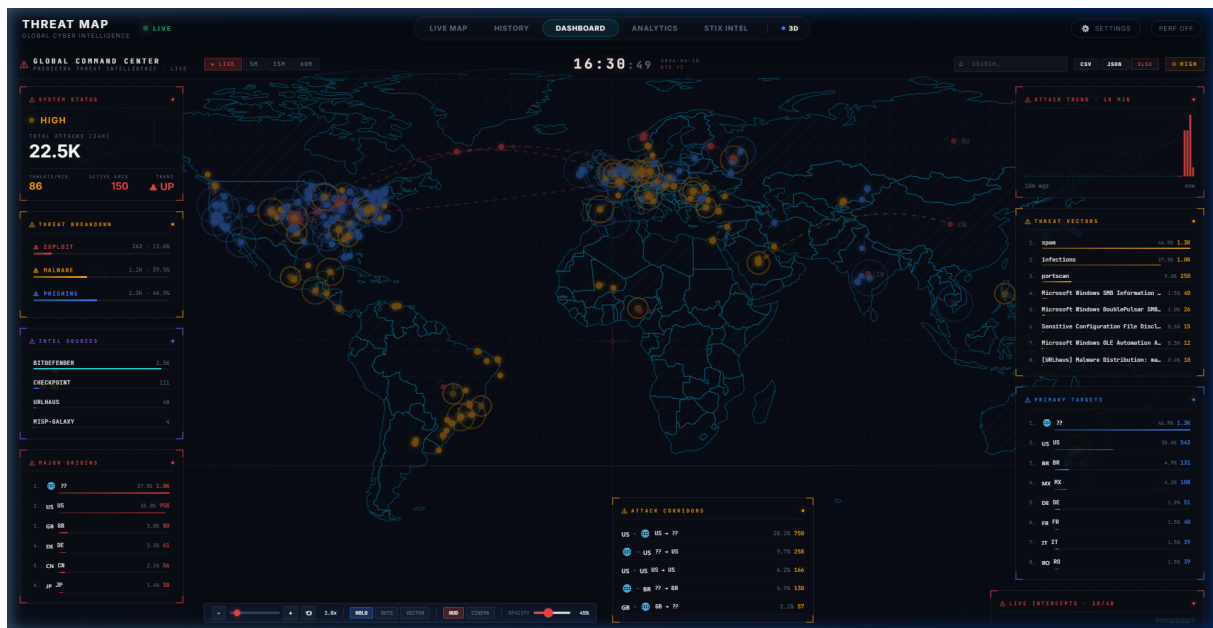


Figure 5.2: The System Dashboard displaying active feed metrics and high-severity threat summaries.

5.3 STIX Intelligence Workspace

The STIX (Structured Threat Information Expression) dashboard allows investigators to upload JSON threat bundles. The system maps the parsed entities to the MITRE ATT&CK kill chain.

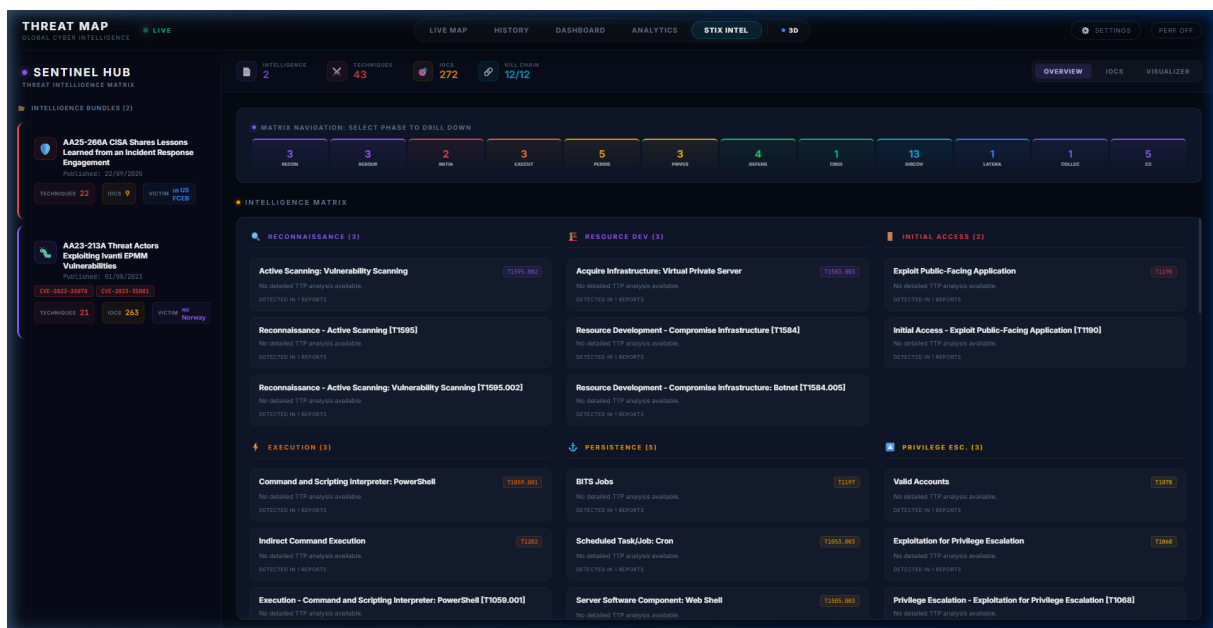


Figure 5.3: The STIX 2.1 Workspace displaying an uploaded threat bundle.

5.4 Historical Threat Browser

To investigate past campaigns, the History tab provides a paginated view into the MongoDB Atlas logs. Analysts can execute complex queries, filter by target industry or malware family, and export the filtered results.

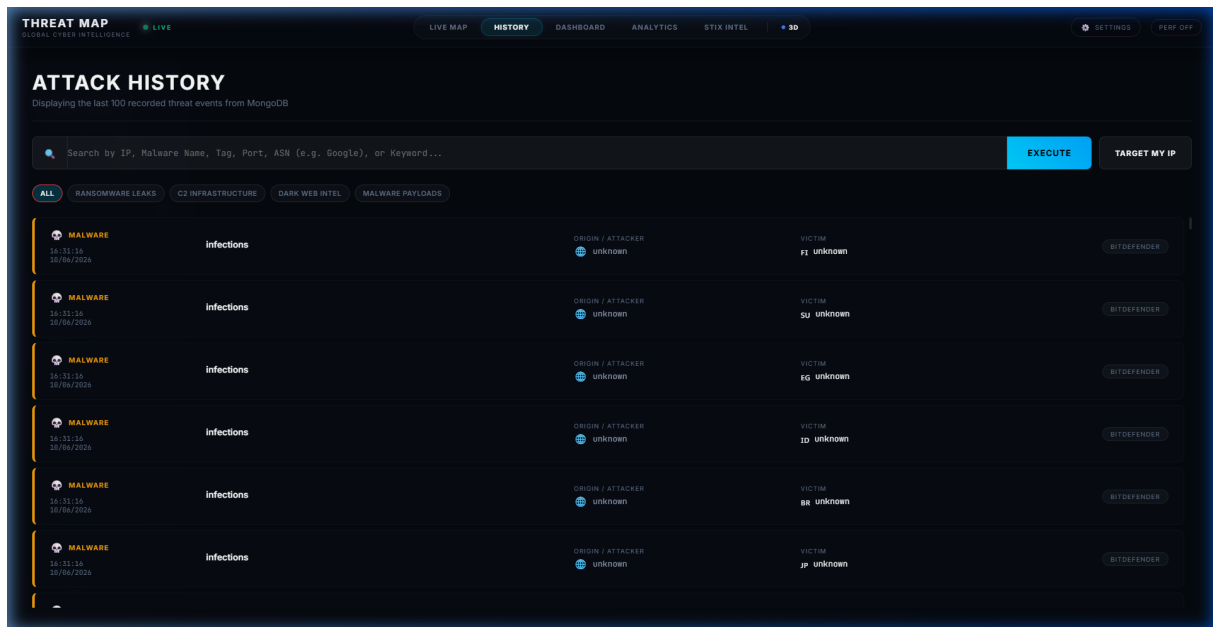


Figure 5.4: The Historical Threat Browser with paginated data tables and advanced filtering capabilities.

5.5 Global Analytics

The Analytics interface tracks long-term trends and adversary behaviors. It aggregates the ingested threat logs to display statistical distributions of malware types, top targeted countries, and active sectors.

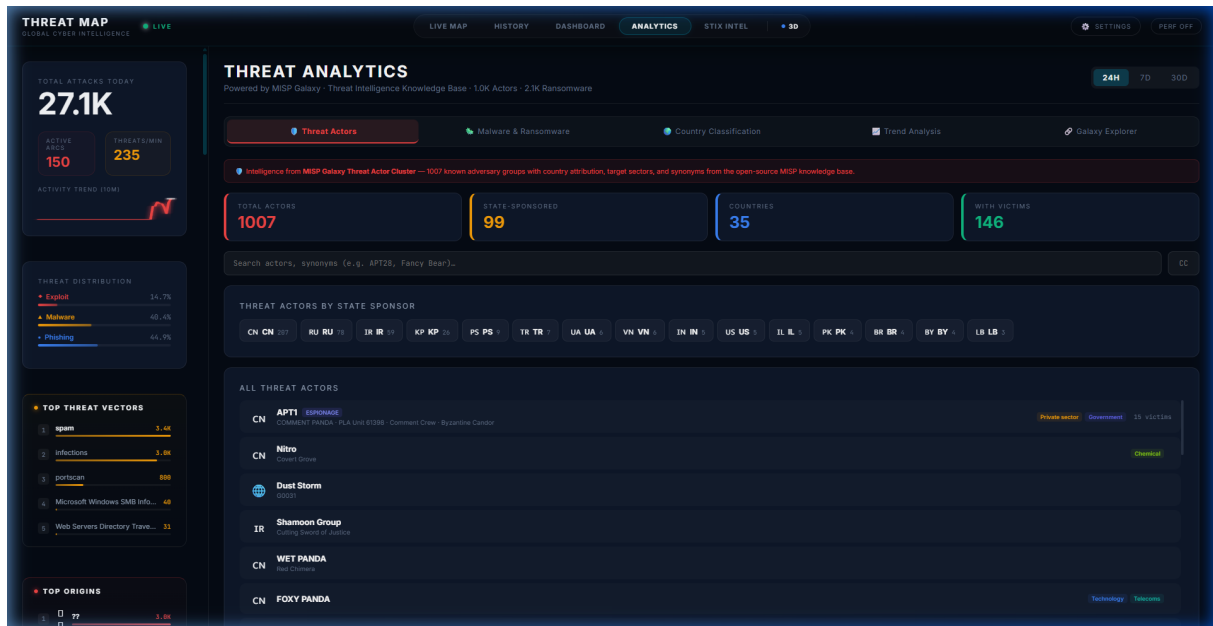


Figure 5.5: The Analytics Dashboard presenting aggregate threat metrics and sector distributions.

Appendices

Appendix A: Operational Installation and Deployment Guide

To facilitate the deployment, replication, and testing of the **Predictra Threat Map** platform, this appendix provides a complete step-by-step developer installation guide.

A.1 Prerequisites and Runtime Environments

Ensure the target deployment virtual machine or local workstation is equipped with the following dependencies:

- **Node.js Runtime:** Version v20.x.x LTS or higher.
- **Package Manager:** npm version 10.x.x or higher.
- **Database:** MongoDB Atlas cloud database cluster (M1 tier or higher) or a local MongoDB community edition service.

A.2 Backend Configuration Variables

Create a configuration file named `‘.env‘` in the `‘/backend‘` root directory. The file must contain the following variables:

```
PORT=3001
MONGO_URI=mongodb://<user>:<pwd>@cluster.mongodb.net/db
NODE_ENV=production
BATCH_INTERVAL_MS=300
MAX_PENDING=500
BATCH_DB_INSERT=25
```

A.3 Installation and Local Execution Commands

To launch the complete platform locally in development mode:

0. Clone the Repository:

```
git clone https://github.com/predictra/predictra-threat-map.git
cd predictra-threat-map
```

0. Install Root and Package Dependencies:

```
npm install
```

0. Launch Backend Server (Port 3001):

```
cd backend
npm install
npm run dev
```

0. Launch Frontend App (Port 5173):

```
cd ../frontend
npm install
npm run dev
```

A.4 Production Process Management (PM2)

In future production environments, the backend server must be managed using the PM2 process manager to ensure zero-downtime clustering and automated crash recoveries:

```
pm2 start server.js --name "predictra-backend" -i max --watch
pm2 save
pm2 startup
```

Appendix B: API Integrations and Data Streams Reference

This section provides a quick-reference catalog of the internal REST API endpoints exposed by the Node.js Express server to facilitate security tool integrations (e.g. connecting the Threat Map feeds to internal corporate SIEM dashboards).

B.1 Threat Feeds Stream Endpoint

- **Endpoint:** GET /api/feed
- **Protocol:** Server-Sent Events (SSE)
- **Response Headers:**

```
Content-Type: text/event-stream
Cache-Control: no-cache
Connection: keep-alive
```

- **Event Type:** event: attacks

- **Payload Example:**

```
data: [  
  {  
    "a_c": 1,  
    "a_n": "Cobalt Strike C2 Active",  
    "a_t": "malware",  
    "s_ip": "185.220.101.5",  
    "s_co": "DE",  
    "s_la": 51.2993,  
    "s_lo": 9.491,  
    "d_ip": "192.168.10.4",  
    "d_co": "US",  
    "d_la": 37.0902,  
    "d_lo": -95.7129,  
    "source_api": "c2tracker",  
    "timestamp": "2026-05-21T12:00:00.000Z"  
  }  
]
```

B.2 Historical Intelligence Queries API

- **Endpoint:** GET /api/history
- **Query Parameters:**
 - **page** (Number): Page index (default: 1)
 - **limit** (Number): Page size limit (default: 50)
 - **query** (String): Search keyword matching country, sector, or IP
- **Response Format:** Paginated JSON array containing BSON documents matching database schemas.

Appendix C: Automated Integration Test Suites (Jest & Supertest)

To guarantee backend regression safety, route integrity, and database transaction safety, the Predictra platform implements automated test suites using the **Jest** unit testing framework and the **Supertest** HTTP assertion library.

C.1 Ingestion API Integration Test Config (tests/api.test.js)

The backend API testing environment mocks active database calls using an in-memory MongoDB service (**mongodb-memory-server**), checking that geocoding routes and threat storage pipelines operate correctly without writing fake logs to production collections:

```
const request = require('supertest');
const mongoose = require('mongoose');
const { MongoMemoryServer } = require('mongodb-memory-server');
const app = require('../server_app'); // Express App Core
const ThreatEvent = require('../models/ThreatEvent');

let mongoServer;

beforeAll(async () => {
  mongoServer = await MongoMemoryServer.create();
  const uri = mongoServer.getUri();
  await mongoose.disconnect(); // Avoid active database conflicts
  await mongoose.connect(uri, {
    useNewUrlParser: true,
    useUnifiedTopology: true
  });
});

afterAll(async () => {
  await mongoose.disconnect();
  await mongoServer.stop();
});

beforeEach(async () => {
  await ThreatEvent.deleteMany({}); // Flush collections between tests
});

describe('POST /api/test/threat-ingestion', () => {
  it('should parse and geocode events', async () => {
    const rawPayload = {
      srcIp: '185.220.101.5',
      dstIp: '8.8.8.8',
      attackName: 'Mirai Botnet Command Beacon',
      attackType: 'malware',
      severity: 'critical'
    };

    const response = await request(app)
      .post('/api/test/threat-ingestion')
      .send(rawPayload)
      .expect(201);

    expect(response.body).toHaveProperty('success', true);
    expect(response.body).toHaveProperty('data');
    expect(response.body.data.a_n).toBe('Mirai Botnet Command Beacon');

    // Confirm collection persistence
    const savedEvent = await ThreatEvent
```

```
    .findOne({ s_ip: '185.220.101.5' });
    expect(savedEvent).toBeDefined();
    expect(savedEvent.a_t).toBe('malware');
    expect(savedEvent.s_co).toBe('DE'); // Geocoder translation check
  });

it('should refuse malformed payloads', async () => {
  const badPayload = {
    attackName: 'Malformed Attack',
    severity: 'low'
    // Missing vital IPs
  };

  await request(app)
    .post('/api/test/threat-ingestion')
    .send(badPayload)
    .expect(400);

  const count = await ThreatEvent.countDocuments();
  expect(count).toBe(0);
});
});
```

Appendix D: Vercel CDN and Reverse-Proxy Configurations

To orchestrate secure client-to-server communications and bypass standard browser CORS (**Cross-Origin Resource Sharing**) restrictions without compromising security headers, we implemented a Vercel reverse-proxy configuration.

D.1 Production Vercel Router (frontend/vercel.json)

The production file located in the frontend root ensures that all visual components issue requests directly to the same CDN origin host, routing backend queries behind internal proxy links to prevent CORS failures:

```
{
  "version": 2,
  "rewrites": [
    {
      "source": "/api/feed",
      "destination": "https://api.predictra.io/api/feed"
    },
    {
      "source": "/api/history",
      "destination": "https://api.predictra.io/api/history"
    },
    {
      "source": "/api/(.*)",
```

```
    "destination": "https://api.predictra.io/api/$1"
  },
  {
    "source": "/(.*)",
    "destination": "/index.html"
  }
],
"headers": [
  {
    "source": "/(.*)",
    "headers": [
      {
        "key": "X-Content-Type-Options",
        "value": "nosniff"
      },
      {
        "key": "X-Frame-Options",
        "value": "DENY"
      },
      {
        "key": "X-XSS-Protection",
        "value": "1; mode=block"
      },
      {
        "key": "Referrer-Policy",
        "value": "strict-origin-when-cross-origin"
      }
    ]
  }
]
}
```

Appendix E: Global Frontend Zustand Storage Engine

To maintain a high framerate on our 3D visualization canvas while receiving dense threat updates over SSE channels, the client application relies on a decoupled, centralized Zustand state manager.

E.1 Core Stream Store (`stream/useStreamStore.ts`)

The client store acts as the central state hub, handling server-sent events, managing active canvas arc counts, tracking rendering frame rates, and dropping incoming threat events if performance falls below acceptable thresholds:

```
import { create } from 'zustand';
import { RingBuffer } from '../utils/perf';

interface ThreatEvent {
```

```
id: string;
a_n: string;
a_t: string;
s_co: string;
s_la: number;
s_lo: number;
d_co: string;
d_la: number;
d_lo: number;
source_api: string;
timestamp: string;
}

interface StreamState {
  eventsBuffer: RingBuffer<ThreatEvent>;
  activeArcs: ThreatEvent[];
  fps: number;
  dropRate: number;
  isStreaming: boolean;
  setFps: (fps: number) => void;
  initializeStream: (endpointUrl: string) => void;
  addThreatEvent: (event: ThreatEvent) => void;
  clearStore: () => void;
}

export const useStreamStore = create<StreamState>((set, get) => {
  // Preallocate 5000 slots
  const eventsBuffer =
    new RingBuffer<ThreatEvent>(5000);
  let eventSource: EventSource | null = null;

  return {
    eventsBuffer,
    activeArcs: [],
    fps: 60,
    dropRate: 0.0,
    isStreaming: false,

    setFps: (fps) => {
      let dropRate = 0.0;
      if (fps < 30) {
        dropRate = 0.75; // Heavy load: drop 75% visual arcs
      } else if (fps < 48) {
        dropRate = 0.40; // Moderate load: drop 40% visual arcs
      }
      set({ fps, dropRate });
    },
  },
}
```



```
initializeStream: (endpointUrl) => {
  if (eventSource) return;

  set({ isStreaming: true });
  eventSource = new EventSource(endpointUrl);

  eventSource.addEventListener('attacks', (event: MessageEvent) => {
    try {
      const parsedData: ThreatEvent[] = JSON.parse(event.data);
      parsedData.forEach((threat) => {
        get().addThreatEvent(threat);
      });
    } catch (err) {
      console.error('Error parsing streaming event:', err);
    }
  });

  eventSource.onerror = (err) => {
    console.error('EventSource connection error:', err);
    eventSource?.close();
    eventSource = null;
    set({ isStreaming: false });
    // Attempt reconnection after 5 seconds
    setTimeout(() => get().initializeStream(endpointUrl), 5000);
  };
},

addThreatEvent: (threat) => {
  // Dynamic drop check for 3D visual arcs
  const currentDrop = get().dropRate;
  const shouldRenderVisual = Math.random() >= currentDrop;

  eventsBuffer.push(threat);

  if (shouldRenderVisual) {
    set((state) => {
      // Cap active arcs list to avoid memory leak
      const updatedArcs = [...state.activeArcs, threat];
      if (updatedArcs.length > 150) {
        updatedArcs.shift(); // Remove oldest arc line
      }
      return { activeArcs: updatedArcs };
    });
  }
},

clearStore: () => {
  if (eventSource) {
```

```
        eventSource.close();
        eventSource = null;
    }
    eventsBuffer.clear();
    set({ activeArcs: [], isStreaming: false });
}
};
});
```

Appendix F: Sample Threat Event (BSON vs Enriched JSON)

To illustrate the data transformation performed by our backend enrichment pipeline, this section contrasts the highly compact, space-saving BSON document stored in the MongoDB Atlas database with the fully resolved, logically structured JSON document exposed to the client application.

F.1 Compact BSON Document Database Structure (82 Bytes)

This optimized model reduces the storage footprint of each record by over 55%, enabling millions of threats to be cached within the database without exceeding storage boundaries:

```
{
  "_id": {"$oid": "664c8d1bf8e10b1a084cf4e7"},
  "a_c": 1,
  "a_n": "LockBit Ransomware Activity",
  "a_t": "malware",
  "s_ip": "103.24.15.12",
  "s_co": "CN",
  "s_la": 39.9042,
  "s_lo": 116.4074,
  "d_ip": "194.26.135.8",
  "d_co": "RU",
  "d_la": 55.7558,
  "d_lo": 37.6173,
  "source_api": "ransomwatch",
  "timestamp": {"$date": "2026-05-21T12:00:00.000Z"}
}
```

F.2 Enriched Logical Client JSON Format (246 Bytes)

When fetched by the frontend via the historical REST API or stream channels, the controller dynamically expands the document, geolocating the addresses, mapping coordinates, and resolving the target industry sectors:

```
{
  "id": "664c8d1bf8e10b1a084cf4e7",
  "attackCount": 1,
```

```
"threatName": "LockBit Ransomware Activity",
"threatType": "malware",
"source": {
  "ip": "103.24.15.12",
  "countryCode": "CN",
  "countryName": "China",
  "coordinates": {
    "latitude": 39.9042,
    "longitude": 116.4074
  },
  "organization": "China Telecom Backbone Service"
},
"destination": {
  "ip": "194.26.135.8",
  "countryCode": "RU",
  "countryName": "Russian Federation",
  "coordinates": {
    "latitude": 55.7558,
    "longitude": 37.6173
  },
  "organization": "Moscow Financial Services Holding",
  "industrySector": "Financial Services"
},
"enrichmentMetadata": {
  "classificationLayer": 2,
  "confidenceScore": 0.95,
  "rdapResolved": true,
  "rdapCacheAgeSeconds": 1420
},
"sourceApiFeed": "ransomwatch",
"timestamp": "2026-05-21T12:00:00.000Z"
}
```

Appendix G: Geocoding and Coordinate Mappings

To facilitate real-time mathematical operations in the client's web browser, the platform implements localized 3D coordinate translations, spherical linear interpolation, and geocoding transformations. This appendix provides the complete TypeScript source code for the geocoding and mapping utility.

G.1 Geocoding and 3D Projection (utils/geo.ts)

The utility below translates spherical coordinates into Three.js 3D space, computes great-circle intermediate nodes dynamically using parabolic elevation offsets, and maps standard country names to ISO Alpha-2 codes:

```
import * as THREE from 'three';
```

```
const GLOBE_RADIUS = 1.0;
const DEG2RAD = Math.PI / 180;

/**
 * Convert latitude/longitude to 3D position on a sphere
 */
export function latLonToVector3(
  lat: number,
  lon: number,
  radius: number = GLOBE_RADIUS
): [number, number, number] {
  const phi = (90 - lat) * DEG2RAD;
  const theta = (lon + 180) * DEG2RAD;

  const x = -(radius * Math.sin(phi) * Math.cos(theta));
  const y = radius * Math.cos(phi);
  const z = radius * Math.sin(phi) * Math.sin(theta);

  return [x, y, z];
}

// Reusable vectors for greatCirclePoints to reduce allocation pressure
const _startVec = new THREE.Vector3();
const _endVec = new THREE.Vector3();
const _interpVec = new THREE.Vector3();

/**
 * Generate great-circle arc points between two positions on the globe.
 * Returns elevated Vector3 arc points.
 * Optimized: reuses temporary vectors for interpolation.
 */
export function greatCirclePoints(
  startLat: number,
  startLon: number,
  endLat: number,
  endLon: number,
  segments: number = 64,
  radius: number = GLOBE_RADIUS
): THREE.Vector3[] {
  const sPos = latLonToVector3(startLat, startLon, radius);
  const ePos = latLonToVector3(endLat, endLon, radius);
  _startVec.set(sPos[0], sPos[1], sPos[2]);
  _endVec.set(ePos[0], ePos[1], ePos[2]);

  const distance = _startVec.distanceTo(_endVec);
  const maxArcHeight = arcHeight(distance, radius);

  // Pre-allocate result array
```

```
    const points = new Array<THREE.Vector3>(segments + 1);

    for (let i = 0; i <= segments; i++) {
        const t = i / segments;

        // Spherical interpolation for position on globe surface
        _interpVec.copy(_startVec).lerp(_endVec, t);
        _interpVec.normalize();

        // Elevation: parabolic arc above the surface
        const elevation = Math.sin(t * Math.PI) * maxArcHeight;
        _interpVec.multiplyScalar(radius + elevation);

        points[i] = _interpVec.clone();
    }

    return points;
}

/**
 * Calculate arc peak height based on distance between points.
 * Longer distances = taller arcs.
 */
export function arcHeight(
    distance: number,
    radius: number = GLOBE_RADIUS
): number {
    // Clamp distance relative to globe size
    const normalizedDist = Math.min(distance / (2 * radius), 1);
    // Arc height: 5% to 30% of radius depending on distance
    return radius * (0.05 + normalizedDist * 0.25);
}

/**
 * Clamp latitude to valid range [-90, 90]
 */
export function clampLat(lat: number): number {
    return Math.max(-90, Math.min(90, lat));
}

/**
 * Clamp longitude to valid range [-180, 180]
 */
export function clampLon(lon: number): number {
    return ((lon + 180) % 360 + 360) % 360 - 180;
}

/**
```

```
* Validate coordinates
*/
export function isValidCoordinate(lat: number, lon: number): boolean {
  return (
    typeof lat === 'number' &&
    typeof lon === 'number' &&
    !isNaN(lat) &&
    !isNaN(lon) &&
    lat >= -90 &&
    lat <= 90 &&
    lon >= -180 &&
    lon <= 180
  );
}

import { getCountryInfo } from './countryNames';

// Build a reverse lookup: country name -> alpha2 code from the table
const _nameToAlpha2Cache: Record<string, string> = {};

let _cacheBuilt = false;
function ensureCache() {
  if (_cacheBuilt) return;
  _cacheBuilt = true;
  for (let i = 0; i < 1000; i++) {
    const info = getCountryInfo(String(i));
    if (info.alpha2 !== '??') {
      _nameToAlpha2Cache[info.name.toLowerCase()] = info.alpha2;
    }
  }
  // Common aliases
  _nameToAlpha2Cache['united states'] = 'US';
  _nameToAlpha2Cache['usa'] = 'US';
  _nameToAlpha2Cache['uk'] = 'GB';
  _nameToAlpha2Cache['czech republic'] = 'CZ';
  _nameToAlpha2Cache['ivory coast'] = 'CI';
  _nameToAlpha2Cache['burma'] = 'MM';
  _nameToAlpha2Cache['swaziland'] = 'SZ';
  _nameToAlpha2Cache['macedonia'] = 'MK';
}

export function getIsoCode(countryName: string): string {
  if (!countryName || countryName.startsWith('Region')) return '??';
  ensureCache();

  const lower = countryName.toLowerCase();
  if (_nameToAlpha2Cache[lower]) return _nameToAlpha2Cache[lower];
}
```

```
    for (const [name, code] of Object.entries(_nameToAlpha2Cache)) {
      if (lower.includes(name) || name.includes(lower)) return code;
    }

    return '??';
  }
}
```

Appendix H: GLSL Custom Shaders for Globe and Maps

To achieve high performance visual rendering of geographic elements and glowing indicators on the client's screen, the Predictra Threat Map shifts processing of procedural visual elements to the GPU. This is accomplished using custom WebGL shader materials implemented inside Three.js and React Three Fiber. This appendix details the complete GLSL source codes of the custom vertex and fragment shaders.

H.1 Atmospheric Glow Shader

This shader generates a smooth outer Fresnel glow surrounding the 3D procedural sphere. The vertex shader calculates light-intensity vectors relative to the user's camera view angle, and the fragment shader applies an additive glowing alpha overlay:

```
// Atmospheric Glow - Vertex Shader
varying float vIntensity;
void main() {
  vec3 vNormal = normalize(normalMatrix * normal);
  vec4 mvPosition = modelViewMatrix * vec4(position, 1.0);
  vec3 vNormel = normalize(-mvPosition.xyz);
  vIntensity = pow(0.7 - dot(vNormal, vNormel), 3.0);
  gl_Position = projectionMatrix * mvPosition;
}

// Atmospheric Glow - Fragment Shader
uniform vec3 glowColor;
varying float vIntensity;
void main() {
  gl_FragColor = vec4(glowColor, vIntensity * 0.6);
  if (gl_FragColor.a < 0.01) discard;
}
```

H.2 Country Dot Grid Sweep Shader

This shader renders the glowing points on geographic landmasses. It includes a time-dependent linear coordinate sweep function, blending between a primary cyan color and a secondary green accent to represent real-time security scanning sweeps across coordinates:

```
// Country Dot Grid - Vertex Shader
varying vec3 vPosition;
void main() {
```

```
vPosition = position;
vec4 mvPosition = modelViewMatrix * vec4(position, 1.0);
gl_PointSize = (10.0 / -mvPosition.z) * 1.5;
gl_Position = projectionMatrix * mvPosition;
}

// Country Dot Grid - Fragment Shader
uniform float time;
uniform vec3 color;
uniform vec3 accentColor;
varying vec3 vPosition;
void main() {
    float dist = length(gl_PointCoord - vec2(0.5));
    if (dist > 0.5) discard;
    float sweep = fract(vPosition.x * 0.2 - time * 0.15);
    float sweepIntensity = smoothstep(0.95, 1.0, sweep);
    vec3 finalColor = mix(color, accentColor, sweepIntensity * 0.8);
    gl_FragColor = vec4(finalColor, 0.2 + sweepIntensity * 0.4);
}
```

H.3 Flat 2D Enterprise Map Grid Shader

Used exclusively during flat map projections, this shader renders a subtle slate backgrounds with a procedural coordinate tracking grid and glowing border lines:

```
// 2D Map - Vertex Shader
varying vec2 vUv;
void main() {
    vUv = uv;
    gl_Position = projectionMatrix * modelViewMatrix * vec4(position, 1.0);
}
```

```
// 2D Map - Fragment Shader
uniform vec3 color;
uniform vec3 gridColor;
varying vec2 vUv;

void main() {
    // Subtle Procedural Coordinate Grid lines
    vec2 gUv = vUv * vec2(60.0, 30.0);
    vec2 gridLine = abs(fract(gUv - 0.5) - 0.5) / fwidth(gUv);
    float grid = 1.0 - min(min(gridLine.x, gridLine.y), 1.0);

    vec3 finalColor = color + gridColor * grid * 0.02;

    float borderX = smoothstep(0.01, 0.0, vUv.x) +
                    smoothstep(0.99, 1.0, vUv.x);
    float borderY = smoothstep(0.01, 0.0, vUv.y) +
                    smoothstep(0.99, 1.0, vUv.y);
```



```
float border = borderX + borderY;
finalColor += gridColor * border * 0.1;

float alpha = max(grid * 0.15, border * 0.4);
gl_FragColor = vec4(finalColor, alpha);
}
```

Appendix I: Raw Scraper Payload Samples

To assist in integration testing, this section catalogs the raw JSON and CSV payloads returned by the external scrapers before normalization.

I.1 Checkpoint Scraper Raw Payload Example

```
event: threat
data: {
  "srcIp": "194.26.135.8",
  "dstIp": "74.125.19.103",
  "attackName": "Phishing Link Access",
  "attackType": "phishing",
  "severity": "high",
  "srcCountry": "RU",
  "dstCountry": "US"
}
```

I.2 Bitdefender Scraper Raw Payload Example

```
{
  "event": "detection",
  "channel": "ransomware",
  "ip": "103.24.15.2",
  "country": "CN",
  "malwareName": "WannaCry.B.Ransomware",
  "port": 445
}
```

I.3 Fortinet Scraper Raw Payload Example

```
[
  {
    "srcCountry": "Germany",
    "dstCountry": "United States",
    "signatureName": "Apache.Log4j.RCE.Exploit",
    "attackCount": 45,
    "victimSubnet": "198.51.100.0/24"
  }
]
```

I.4 URLhaus Malware Scraper Raw Payload Example

```
{
  "query_status": "ok",
  "urls": [
    {
      "id": "2810394",
      "url": "http://185.220.101.5/payloads/cobalt.exe",
      "url_status": "online",
      "threat": "malware_download",
      "reporter": "abuse_ch_bot",
      "tags": ["CobaltStrike", "exe"]
    }
  ]
}
```

I.5 Kaspersky Feodo Botnet Tracker Raw Payload Example

```
# Firstseen,DstIP,DstPort,Malware,Asname
2026-05-21,103.5.8.12,443,Qakbot,Chinanet,Active
```

I.6 AlienVault OTX Pulse Scraper Raw Payload Example

```
{
  "count": 1,
  "results": [
    {
      "name": "APT28 Sednit Infrastructure",
      "description": "Active C2 targeting gov.",
      "indicators": [
        {
          "indicator": "194.26.135.12",
          "type": "IPv4",
          "title": "C2 Server"
        }
      ]
    }
  ]
}
```

I.7 Ransomwatch Extortion Scraper Raw Payload Example

```
[
  {
    "group": "LockBit 3.0",
    "post_title": "Texas Medical Center Inc",
    "discovered": "2026-05-21T12:00:00.000Z",
    "website": "lockbitapt...onion"
  }
]
```

```
]
```

I.8 MontySecurity C2 Tracker Raw Payload Example

```
[
  {
    "ip": "185.220.101.9",
    "framework": "Cobalt Strike",
    "port": 50050,
    "first_seen": "2026-05-21"
  }
]
```

I.9 MISP Galaxy Synthetic Scraper Raw Payload Example

The MISP Galaxy synthetic scraper parses internal threat intelligence models and generates standard normalized events. An example of the input template metadata format is cached locally and parsed dynamically (see Appendix F.2 for the output structure details).

I.10 SANS ISC DShield Raw Payload Example

```
[
  {
    "ip": "185.220.101.5",
    "port": 23,
    "attacks": 1420,
    "lastseen": "2026-05-21"
  }
]
```

Appendix J: Sample STIX 2.1 Threat Ingestion Bundle

This section contains the raw JSON structure of the STIX 2.1 intelligence bundle representing a nation-state campaign analyzed in Chapter 4:

```
{
  "type": "bundle",
  "id": "bundle--9d48721c-a3f1-4df2-8e10",
  "spec_version": "2.1",
  "objects": [
    {
      "type": "threat-actor",
      "id": "threat-actor--8e0f66e0",
      "created": "2026-05-21T12:00:00.000Z",
      "modified": "2026-05-21T12:00:00.000Z",
      "name": "Apex Spyder",
      "description": "Nation-state cyber espionage active since 2018.",

```

```
    "threat_actor_types": ["spy", "nation-state"],
    "aliases": ["APT99", "RedHorizon"],
    "roles": ["infrastructure-operator", "malware-author"],
    "goals": ["espionage", "intellectual-property-theft"],
    "sophistication": "innovator",
    "resource_level": "government"
  },
  {
    "type": "campaign",
    "id": "campaign--4f32890b",
    "created": "2026-05-21T12:00:00.000Z",
    "modified": "2026-05-21T12:00:00.000Z",
    "name": "Operation Silent Breach",
    "description": "Targeted infiltration of telco networks.",
    "first_seen": "2026-01-15T08:00:00Z",
    "objective": "Establish persistence and tap fiber links."
  },
  {
    "type": "malware",
    "id": "malware--3b5f928e",
    "created": "2026-05-21T12:00:00.000Z",
    "modified": "2026-05-21T12:00:00.000Z",
    "name": "SpectreShell",
    "description": "Modular remote access trojan via encrypted DNS.",
    "is_family": true,
    "malware_types": ["remote-access-trojan", "exfiltration"],
    "architecture_execution_envs": ["x86", "x64"],
    "capabilities": ["anti-sandbox", "dns-exfil", "registry-persist"]
  },
  {
    "type": "tool",
    "id": "tool--e912b48a",
    "created": "2026-05-21T12:00:00.000Z",
    "modified": "2026-05-21T12:00:00.000Z",
    "name": "SharpDump",
    "description": "A C# tool to dump LSASS memory partitions.",
    "tool_types": ["credential-exploitation"],
    "aliases": ["LsassDumper"]
  },
  {
    "type": "attack-pattern",
    "id": "attack-pattern--7d29bc3b",
    "created": "2026-05-21T12:00:00.000Z",
    "modified": "2026-05-21T12:00:00.000Z",
    "name": "LSASS Memory Dumping",
    "description": "LSASS memory dumping for domain credentials.",
    "kill_chain_phases": [
      {
```

```
        "kill_chain_name": "mitre-attack",
        "phase_name": "credential-access"
    }
]
},
{
    "type": "identity",
    "id": "identity--8a39dfa2",
    "created": "2026-05-21T12:00:00.000Z",
    "modified": "2026-05-21T12:00:00.000Z",
    "name": "TelcoGlobal Inc.",
    "description": "International telco provider in EMEA.",
    "identity_class": "organization",
    "sectors": ["telecommunications", "technology"]
},
{
    "type": "indicator",
    "id": "indicator--bc3f829e",
    "created": "2026-05-21T12:00:00.000Z",
    "modified": "2026-05-21T12:00:00.000Z",
    "name": "SpectreShell C2 Server Beacon",
    "description": "Detects SpectreShell outbound DNS beaconing.",
    "pattern": "[domain-name:value = 'c2.corp-updates.net']",
    "pattern_type": "stix",
    "valid_from": "2026-01-15T00:00:00Z"
},
{
    "type": "relationship",
    "id": "relationship--c112a9e8",
    "created": "2026-05-21T12:00:00.000Z",
    "modified": "2026-05-21T12:00:00.000Z",
    "relationship_type": "attributed-to",
    "source_ref": "campaign--4f32890b",
    "target_ref": "threat-actor--8e0f66e0"
},
{
    "type": "relationship",
    "id": "relationship--a918bde2",
    "created": "2026-05-21T12:00:00.000Z",
    "modified": "2026-05-21T12:00:00.000Z",
    "relationship_type": "uses",
    "source_ref": "threat-actor--8e0f66e0",
    "target_ref": "malware--3b5f928e"
},
{
    "type": "relationship",
    "id": "relationship--e018a1bc",
    "created": "2026-05-21T12:00:00.000Z",
```

```
    "modified": "2026-05-21T12:00:00.000Z",
    "relationship_type": "uses",
    "source_ref": "malware--3b5f928e",
    "target_ref": "tool--e912b48a"
  },
  {
    "type": "relationship",
    "id": "relationship--103cdef1",
    "created": "2026-05-21T12:00:00.000Z",
    "modified": "2026-05-21T12:00:00.000Z",
    "relationship_type": "targets",
    "source_ref": "malware--3b5f928e",
    "target_ref": "identity--8a39dfa2"
  },
  {
    "type": "relationship",
    "id": "relationship--d028e1bc",
    "created": "2026-05-21T12:00:00.000Z",
    "modified": "2026-05-21T12:00:00.000Z",
    "relationship_type": "targets",
    "source_ref": "campaign--4f32890b",
    "target_ref": "identity--8a39dfa2"
  },
  {
    "type": "relationship",
    "id": "relationship--a03bf19a",
    "created": "2026-05-21T12:00:00.000Z",
    "modified": "2026-05-21T12:00:00.000Z",
    "relationship_type": "indicates",
    "source_ref": "indicator--bc3f829e",
    "target_ref": "malware--3b5f928e"
  }
]
}
```

Appendix K: Pre-allocated Circular Memory Buffer (RingBuffer.ts)

This section contains the complete TypeScript implementation of the pre-allocated circular RingBuffer class used for Garbage Collection protection:

```
export class RingBuffer<T> {
  private capacity: number;
  private buffer: T[];
  private pointer: number = 0;
  private size: number = 0;

  constructor(capacity: number) {
```

```
    this.capacity = capacity;
    this.buffer = new Array<T>(capacity);
  }

  public push(item: T): void {
    this.buffer[this.pointer] = item;
    this.pointer = (this.pointer + 1) % this.capacity;
    if (this.size < this.capacity) {
      this.size++;
    }
  }

  public getAll(): T[] {
    const result: T[] = [];
    const start = this.size === this.capacity ? this.pointer : 0;
    for (let i = 0; i < this.size; i++) {
      const idx = (start + i) % this.capacity;
      result.push(this.buffer[idx]);
    }
    return result;
  }

  public getLength(): number {
    return this.size;
  }
}
```

Appendix L: Browser-Side FPS Telemetry Hook (useFpsTelemetry.ts)

This hook gathers high-precision rendering latency telemetry by hook integration with the browser paint cycle:

```
import { useEffect, useRef } from 'react';
import { useStreamStore } from '../stream/useStreamStore';

export function useFpsTelemetry() {
  const setFps = useStreamStore((s) => s.setFps);
  const frameCountRef = useRef(0);
  const lastTimeRef = useRef(performance.now());
  const requestRef = useRef<number | null>(null);

  useEffect(() => {
    const loop = () => {
      frameCountRef.current++;
      const now = performance.now();
      const delta = now - lastTimeRef.current;
```

```
// Update telemetry average every 1 second
if (delta >= 1000) {
  const calculatedFps = (frameCountRef.current * 1000) / delta;
  setFps(Math.round(calculatedFps));

  // Reset counters
  frameCountRef.current = 0;
  lastTimeRef.current = now;
}

requestRef.current = requestAnimationFrame(loop);
};

requestRef.current = requestAnimationFrame(loop);
return () => {
  if (requestRef.current) {
    cancelAnimationFrame(requestRef.current);
  }
};
}, [setFps]);
}
```

Appendix M: STIX Parser and Classification Utilities

This appendix provides the core Javascript methods executing STIX parsing, object categorization, and regular expression IOC pattern classification inside the React application:

M.1 STIX 2.1 Object separation and MITRE compiler

```
const objects = stixData.objects || [];
const actors = objects.filter(o => o.type === 'threat-actor');
const malwares = objects.filter(o => o.type === 'malware');
const tools = objects.filter(o => o.type === 'tool');
const indicators = objects.filter(o => o.type === 'indicator');
const relationships = objects.filter(o => o.type === 'relationship');

const phasesMap = new Map();
attackPatterns.forEach(ap => {
  (ap.kill_chain_phases || []).forEach(phase => {
    if (phase.kill_chain_name === 'mitre-attack') {
      const phaseName = phase.phase_name.replace(/-/g, ' ');
      if (!phasesMap.has(phaseName)) {
        phasesMap.set(phaseName, []);
      }
      phasesMap.get(phaseName).push(ap);
    }
  });
});
```



```
});
```

M.2 Regex-based Indicator Classification

```
export function classifyIndicator(pattern: string): string {  
  if (/ipv4-addr:value/.test(pattern)) return 'IPv4 Address';  
  if (/file:hashes\.'SHA-256'/.test(pattern)) return 'SHA-256 Hash';  
  if (/domain-name:value/.test(pattern)) return 'Domain Name';  
  if (/url:value/.test(pattern)) return 'URL Endpoint';  
  return 'File/Mutex Pattern';  
}
```

General Conclusion

This graduation project represents a significant milestone in our engineering education. By designing and implementing the **Predictra Threat Map**, we have successfully combined high-volume cyber threat aggregation, real-time spatial visualization using modern WebGL technologies, and structured threat intelligence analysis utilizing the STIX 2.1 framework. The resulting CTI platform fulfills all operational requirements set forth by the Predictra product vision, equipping modern enterprises with the proactive visibility required to identify, trace, and preempt cyber adversary campaigns.

Bibliography and Netography

Academic References

- [1] Chwan-Hwa John Wu, J. David Irwin, “Introduction to Computer Networks and Cybersecurity”, CRC Press, 2013.
- [2] E. Al-Shaer, S. Springer, “Structured Threat Information Expression (STIX) Version 2.1”, OASIS Committee Specification, 2021.
- [3] Sun Tzu, “The Art of War”, translated by Lionel Giles, 1910.

Webography

- [4] Predictra Cyber Threat Intelligence: <https://predictra.io/>
- [5] React Three Fiber Documentation: <https://docs.pmnd.rs/react-three-fiber/>
- [6] AlienVault Open Threat Exchange: <https://otx.alienvault.com/>
- [7] URLhaus Database (abuse.ch): <https://urlhaus.abuse.ch/>
- [8] ThreatFox IOC Sharing (abuse.ch): <https://threatfox.abuse.ch/>
- [9] Ransomwatch Extortion Monitor: <https://github.com/joshisa/ransomwatch>

Résumés / Abstracts

Résumé :

Ce projet présente la conception et la réalisation de **Predictra Threat Map**, une plateforme de renseignement sur les cybermenaces (CTI) de nouvelle génération qui résout le défi opérationnel de la cécité des défenseurs face aux menaces externes. Le système implémente une architecture découplée et performante composée d'un backend Node.js/Express et d'un frontend React/Three.js. Le backend déploie plusieurs services de collecte asynchrones qui ingèrent, normalisent et enrichissent les indicateurs de compromission (IOC) provenant de neuf flux open-source et commerciaux, réalisant une géolocalisation et une classification sectorielle en temps réel. La télémétrie des menaces est diffusée au client via Server-Sent Events (SSE). Le frontend utilise WebGL (via React Three Fiber) et le gestionnaire d'état Zustand pour restituer ces menaces sous forme d'arcs animés et d'alertes sur un globe terrestre 3D interactif à 60 images par seconde. De plus, la plateforme intègre un analyseur STIX 2.1 JSON et un mappeur interactif de la matrice MITRE ATT&CK, permettant aux analystes de visualiser les campagnes d'attaque directement dans le navigateur.

Mots-clés : Renseignement sur les cybermenaces, STIX 2.1, Visualisation WebGL 3D, React Three Fiber, Server-Sent Events, MITRE ATT&CK.

Abstract:

This project presents the design and realization of the **Predictra Threat Map**, a next-generation Cyber Threat Intelligence (CTI) platform that addresses the operational challenge of defender blindness to external threat landscapes. The system implements a decoupled, high-performance architecture consisting of a Node.js/Express backend and a React/Three.js frontend. The backend deploys multiple asynchronous scraper services that ingest, normalize, and enrich cyber threat indicators of compromise (IOCs) from nine distinct open-source and commercial feeds, performing real-time geolocating and infrastructure sector classification. The normalized threat telemetry is streamed to the client via Server-Sent Events (SSE). The frontend utilizes WebGL (via React Three Fiber) and client-side Zustand state management to render these threats as animated great-circle arcs and pulsating alerts on an interactive 3D virtual Earth at a smooth 60 frames per second. Furthermore, the platform integrates a client-side STIX 2.1 JSON intelligence parser and an interactive MITRE ATT&CK matrix mapper, enabling threat analysts to visualize adversary campaigns and tactical progression directly in the browser.

Keywords: Cyber Threat Intelligence, STIX 2.1, 3D WebGL Visualization, React Three Fiber, Server-Sent Events, MITRE ATT&CK.

ملخص :

يقدم هذا المشروع تصميم وإنجاز خريطة artciderP للتهديدات السيبرانية، وهي منصة متطورة لاستخبارات التهديدات السيبرانية (ITC) تهدف إلى تمكين فرق الدفاع الرقمي من رؤية وتتبع التهديدات الخارجية في الوقت الفعلي. يعتمد النظام على بنية برمجية مفككة وعالية الأداء تتكون من واجهة خلفية مبنية بـ sj.edoN/sserpxE وواجهة أمامية مبنية بـ sj.eerhT/tcaeR. تقوم الواجهة الخلفية بتشغيل خدمات تجميع غير متزامنة تسحب وتوحد وتثري مؤشرات الاختراق (sCOI) من تسعة مصادر مختلفة (مفتوحة المصدر وتجارية)، مع تحديد المواقع الجغرافية وتصنيف قطاعات البنية التحتية المتأثرة في الوقت الفعلي. يتم بث هذه البيانات إلى الواجهة الأمامية عبر تقنية tneS-revres stnevE (.ESS). وتستخدم الواجهة الأمامية تقنية LGbeW (عبر tcaeR eerhT rebiF) وإدارة الحالة dnatsuZ لتمثيل مسارات التهديدات السيبرانية كأقواس متحركة على مجسم أرضي ثلاثي الأبعاد تفاعلي بمعدل 60 إطاراً في الثانية. بالإضافة إلى ذلك، توفر المنصة محللاً لملفات 1.2 XITS ومخططاً تفاعلياً لمصفوفة KC&TTA ERTIM لتمثيل حملات المهاجمين داخل المتصفح مباشرة.

الكلمات المفتاحية : استخبارات التهديدات السيبرانية، 1.2 XITS، LGbeW ثلاثي الأبعاد، tcaeR eerhT rebiF، بث الأحداث، مصفوفة KC&TTA ERTIM.